

A Project Report

On

**ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL
LEARNING ADAPTIVE GRADIENT- SENSITIVE MEMORY CONSOLIDATION**

Submitted

in partial fulfillment for the award of the degree of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE)

by

22F61A3143	-	L.SUMITHRA
22F61A3148	-	GANGALAPUDI VENKATESH
22F61A3127	-	A.POORNIMA
22F61A3153	-	GADIMUTHAKA YOGESH

Under the esteemed guidance of

Dr. J SRIDHAR, M.E., Ph.D.,

Associate Professor, Department of CSE(CCC)



**Department of Computer Science and Engineering
(Artificial Intelligence)**

**SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)**

(Approved by AICTE & Affiliated to JNTU, Anantapuram)

(Accredited by NBA for Civil, EEE, ECE, MECH & CSE)

(Accredited by NAAC with 'A+' Grade)

Siddharth Nagar, Narayanavanam Road, Puttur-517583, A.P

2026

SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)

(Approved by AICTE & Affiliated to JNTU, Anantapuram)

(Accredited by NBA for Civil, EEE, ECE, MECH & CSE)

(Accredited by NAAC with 'A+' Grade)

Siddharth Nagar, Narayanavanam Road, Puttur-517583, A.P

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)



BONAFIDE CERTIFICATE

**Certified that this project report titled “ENHANCED PRIVACY-PRESERVING
FEDERATED CLASS INCREMENTAL LEARNING ADAPTIVE GRADIENT-
SENSITIVE MEMORY CONSOLIDATION ” is a bonafied work of**

22F61A3143	-	L.SUMITHRA
22F61A3148	-	GANGALAPUDI VENKATESH
22F61A3127	-	A.POORNIMA
22F61A3153	-	GADIMUTHAKA YOGESH

in IV B.Tech II Semester of **Computer Science and Engineering (Artificial Intelligence)**. The results embodied in this project report have not been submitted to any other university for award of any degree.

Internal Guide

Dr. J SRIDHAR, M.E., Ph.D.,
Associate Professor,
Department of CSE(CCC),
SIETK.

Head of the Department

Dr. E. Murali M.Tech., Ph.D.,
HOD and Professor,
Department of CSE(AIMD),
SIETK.

Submitted for the main Project viva-voce examination held on _____

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

ACKNOWLEDGEMENT

We take this opportunity to acknowledge all the people who helping us to do our project a successful one.

We greatly convey our sincere thanks to our beloved chairman **Dr. K. Ashok Raju**, M.A, Ph.D., and Vice chairperson **Dr. K. Indiraveni**, M.Tech., Ph.D., for providing us the facilities and time for accomplishing the Project work.

We wish to express our profound gratitude to one of our all-time well-wisher and Principal of our college **Dr. Bogala Madhu**, M.Tech., Ph.D., for his constant encouragement for completing the Project work successfully.

We wish to thank our Head of the Department, **Dr. E. Murali**, M.Tech, Ph.D., for her guidance in accomplishing the Project work.

We wish to convey our heartfelt thanks to **Dr. J SRIDHAR**, M.E., Ph.D., Super- visor of this project and **Dr. N BABU**, M.E., Ph.D., Project Coordinator, who supported and helped to make this Project work as successful one.

We extend our thanks to all the faculty members of the CSE Department and Lab technicians, who gave us the moral support for the completion of the Project work.

We also extend our thanks to our parents and our friends for the encouragement in proceeding the Project work in right way and for the completion of the Project work in successful way.

LIST OF CONTENTS

CHAPTER NO	NAME OF THE CONTENTS	PAGE NO
	BONAFIDE CERTIFICATE	i
	ACKNOWLEDGEMENT	ii
	LIST OF CONTENTS	iii
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
	LIST OF ABBREVIATIONS	ix
	ABSTRACT	x
CHAPTER 1	INTRODUCTION	1-4
	1.1 BACKGROUND	1
	1.2 FEDERATED CLASS-INCREMENTAL LEARNING	1
CHAPTER 2	LITERATURE SURVEY	5-13
	2.1 MACHINE LEARNING	5
	2.2 FEDERATED LEARNING FUNDAMENTALS	10
	2.3 FEDERATED LEARNING WITH INTERMITTENT AND HETEROGENEOUS CLIENTS	10
	2.4 DECENTRALIZED AND EDGE-BASED FEDERATED LEARNING	11
	2.5 ASYNCHRONOUS AND DISTRIBUTED OPTIMIZATION IN FL	11
	2.6 FEDERATED OPTIMIZATION UNDER SYSTEM HETEROGENEITY	12
	2.7 PRIVACY AND SECURITY IN FEDERATED LEARNING	12
	2.8 PRIVACY ATTACKS ON MACHINE LEARNING MODELS	12
	2.9 SUMMARY AND RESEARCH GAP	12

CHAPTER 3	SYSTEM ANALYSIS	15-20
	3.1 INTRODUCTION	15
	3.2 EXISTING SYSTEM ANALYSIS	15
	3.3 PROBLEM STATEMENT	15
	3.4 PROPOSED SYSTEM OVERVIEW	15
	3.5 SYSTEM ARCHITECTURE DESCRIPTION	16
	3.6 FUNCTIONAL REQUIREMENTS	16
	3.7 NON-FUNCTIONAL REQUIREMENTS	16
	3.8 FEASIBILITY ANALYSIS	16
	3.8.1 TECHNICAL&ECONOMIC FEASABILITY	16
	3.8.2 OPERATIONAL FEASIBILITY	17
	3.9 ADVANTAGES OF THE PROPOSED SYSTEM	17
	3.10 USE CASE ANALYSIS	17
	3.11 DATA FLOW ANALYSIS	17
	3.12 CONSTRAINTS OF THE SYSTEM	18
	3.13 RISK ANALYSIS	18
	3.14 PERFORMANCE METRICS CONSIDERED	18
	3.15 DESIGN GOALS	19
	3.16 COMPARISON WITH EXISTING SYSTEM	19
	3.17 ASSUMPTIONS MADE	19
	3.18 VALIDATION STRATEGY	20
	3.19 SUMMARY OF THE CHAPTER	20
CHAPTER 4	SYSTEM DESIGN	21-33
	4.1 INTRODUCTION	21
	4.2 OVERALL SYSTEM ARCHITECTURE	21
	4.3 MODULE-WISE SYSTEM DESIGN	24
	4.3.1 DISTRIBUTED DATA SOURCE MODULE	24
	4.3.2 FEDERATED CLIENT MODULE	25
	4.3.3 RENYI DIFFERENTIAL PRIVACY(RDP) MODULE	25
	4.3.4 ATTENTION-BASED DYNAMIC AGGREGATION MODULE	25
	4.3.5 DUAL-MODEL ARCHITECTURE WITH ELASTIC WEIGHT CONSOLIDATION	26
	4.3.6 MEMORY MANAGEMENT MODULE (CORESET SELECTION)	26
	4.3.7 GLOBAL MODEL UPDATE AND	26

	DISTRIBUTION MODULE	
	4.4 UML DIAGRAM DESIGN	26
	4.4.1 USE CASE DIAGRAM	26
	4.4.2 CLASS DIAGRAM	27
	4.4.3 SEQUENCE DIAGRAM	28
	4.4.4 ACTIVITY DIAGRAM	29
	4.5 DATA FLOW DIAGRAM	30
	4.6 SECURITY AND PRIVACY DESIGN	30
	4.7 DESIGN CONSTRAINTS AND TRADE-OFFS	31
	4.8 DESIGN OBJECTIVES ACHIEVED	31
	4.9 INPUT AND OUTPUT DESIGN	31
CHAPTER 5	SYSTEM IMPLEMENTATION	34-53
	5.1 INTRODUCTION	34
	5.2 OVERALL IMPLEMENTATION WORKFLOW	34
	5.3 MODULE-WISE IMPLEMENTATION	34
	5.3.1 FEDERATED CLIENT IMPLEMENTATION	34
	5.3.2 LOCAL TRAINING AND CLASS- INCREMENTAL LEARNING	35
	5.3.3 RENEYI DIFFERENTIAL PRIVACY IMPLEMENTATION	35
	5.3.4 SECURE COMMUNICATION IMPLEMENTATION	35
	5.3.5 ATTENTION-BASED DYNAMIC AGGREGATION IMPLEMENTATION	35
	5.3.6 DUAL-MODEL ARCHITECTURE WITH EWC IMPLEMENTATION	35
	5.3.7 CORESET MEMORY MANAGEMENT IMPLEMENTATION	35
	5.4 ALGORITHM DESCRIPTION	36
	5.5 PERFORMANCE MONITORING	36
	5.6 IMPLEMENTATION CHALLENGES	36
	5.7 IMPLEMENTATION ENVIRONMENT	37
CHAPTER 6	SYSTEM TESTING	54-57
	6.1 INTRODUCTION	54
	6.2 OBJECTIVES OF SYSTEM TESTING	54
	6.3 TESTING METHODOLOGY	54
	6.4 TYPES OF TESTING PERFORMED	54
	6.4.1 UNIT TESTING	54

	6.4.2 INTEGRATION TESTING	55
	6.4.3 SYSTEM TESTING	55
	6.4.4 PERFORMANCE TESTING	55
	6.4.5 SECURITY AND PRIVACY TESTING	55
	6.5 TEST ENVIRONMENT	56
	6.6 TEST CASES	56
	6.7 VALIDATION OF FUNCTIONAL REQUIREMENTS	56
	6.8 SUMMARY OF THE CHAPTER	57
CHAPTER 7	EXPERIMENTAL ANALYSIS	58-71
	7.1 EXECUTION PROCEDURE	58
	7.2 OUTPUT SCREENSHOT AND DESCRIPTION	61
CHAPTER 8	CONCLUSION AND FUTURE SCOPE	72-74
	8.1 CONCLUSION	72
	8.2 FUTURE SCOPE	72
	REFERENCES	73

LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO
4.1	Client-Server Federated Learning Architecture	21
4.3	Module-Wise System Design	24
4.4.1	Use Case Diagram	27
4.4.2	Class Diagram	28
4.4.3	Sequence Diagram	29
4.4.4	Activity Diagram	30
5.7.1	Implementation Environment	38
5.7.2	Google Drive Dashboard Interface	39
5.7.3	Jupyter Notebook Kernel and Code Cells View	40
5.7.4	Client-Server to Cloud Computing Architecture	40
5.7.5	Python Data Visualization in Jupyter Notebook	41
5.7.6	Google Colab Environment for DDSP Audio Processing	42
5.7.7	Internet and Intranet Web Application Framework	43
5.7.8	Cloud-Based Jupyter Notebook Architecture and Runtime Configuration	44
5.7.9	Google Colab Runtime Management Screen	45
5.7.10	Tensorflow Cloud TPU Architecture and Execution Flow	45
5.7.11	Google Colab Drive Mount Interface	46
5.7.12	Google Drive OAuth Authentication in Google Colab	47
5.7.13	Google Colab Editor and Code Cell Layout	48
5.7.14	Changing Runtime Type and Enabling GPU Acceleration in Google Colab	49
5.7.15	Google Colab Runtime Access Blocked Warning	51
5.7.16	Google Drive Home Screen Interface	51
5.7.17	Welcome to Google Colab Dashboard	52
7.1	Federated Class-Incremental Learning Setup	61
7.2	CIFAR-100 Sample Images	62
7.3	Enhanced Privacy-Preserving Federated Class-Incremental Learning Framework	62
7.4	Comparative Performance Evaluation of PP-FCIL	63
7.5	Comparative Tables of Enhanced PP-FCIL Performance	64
7.6	Comparative Evaluation, Ablation Study, and Complexity Analysis	65
7.7	Enhanced PP-FCIL Training Progress Visualization	66
7.8	Intermediate Accuracy Results Across Incremental Tasks	67
7.9	Final Accuracy and Performance Improvement Analysis	68
7.10	Overall Summary and Performance Highlights of Enhanced PP-FCIL	69
7.11	Enhanced PP-FCIL Final Execution Summary	70
7.12	Execution Completion and Generated Outputs	70

LIST OF TABLES

TABLE NO	NAME OF THE TABLE	PAGE NO
2.1	Summary of Literature Survey	13
3.1	Comparision with Existing System	19
5.7	Feature Comparision of ML Notebook Platforms	51
6.6	Sample Test Cases for Federated Learning Model Validation	56

LIST OF ABBREVIATIONS

E-PP-FCIL	Enhanced Privacy-Preserving Federated Class-Incremental Learning
PP-FCIL	Privacy-Preserving Federated Class-Incremental Learning
AGSMC	Adaptive Gradient-Sensitive Memory Consolidation
FL	Federated Learning
CIL	Class-Incremental Learning
IoT	Internet of Things
FCIL	Federated Class-Incremental Learning
DP	Differential Privacy
RDP	Rényi Differential Privacy
EWC	Elastic Weight Consolidation
FL	Federated Learning
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory

ABSTRACT

In this work, we propose an Enhanced Privacy-Preserving Federated Class-Incremental Learning (E-PP-FCIL) framework that builds upon the original PP-FCIL approach by integrating a novel Adaptive Gradient-Sensitive Memory Consolidation (AGSMC) algorithm. AGSMC introduces three key innovations to address challenges in federated learning where clients continuously receive new data classes while ensuring privacy and minimizing catastrophic forgetting. First, it incorporates gradient-sensitivity aware exemplar selection, which prioritizes samples with high gradient magnitudes to enhance learning. Second, temporal importance weighting adjusts the significance of samples based on when they were learned, preserving crucial knowledge over time. Third, cross-client knowledge distillation leverages anonymized feature statistics from other clients to improve local without compromising privacy. Our extensive experiments on CIFAR-100 and ImageNet datasets demonstrate that E-PP-FCIL with AGSMC achieves a 4–8% improvement in average incremental accuracy over the original PP-FCIL while maintaining equivalent privacy guarantees. The AGSMC algorithm is especially effective in scenarios with highly imbalanced data and varying client participation, making the proposed framework well-suited for practical federated learning applications.

KEYWORDS: *Federated Learning (FL), Class-Incremental Learning (CIL) Cross-Client Knowledge Distillation, Catastrophic Forgetting, Data Imbalance Handling, Client Participation Variability.*

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The rapid growth of data-generating devices such as mobile phones, Internet of Things (IoT) sensors, and edge computing systems has led to the widespread adoption of distributed machine learning techniques. Traditional centralized learning approaches require collecting all data at a central server, which raises serious concerns related to data privacy, communication overhead, and regulatory compliance. To address these challenges, Federated Learning (FL) has emerged as a collaborative learning paradigm that enables multiple clients to train a shared global model without transferring raw data from local devices.

While federated learning successfully preserves data privacy, most existing FL frameworks assume that data classes are static and known in advance. However, in real-world applications such as healthcare, smart surveillance, autonomous systems, and recommendation platforms, data is continuously evolving and new classes may appear over time. This dynamic nature of data requires learning systems to adapt incrementally while retaining previously learned knowledge.

1.2 FEDERATED CLASS-INCREMENTAL LEARNING

Federated Class-Incremental Learning (FCIL) extends traditional federated learning by enabling the model to learn new classes incrementally across distributed clients. In FCIL, different clients may observe new classes at different times, and the global model must integrate this new knowledge without degrading performance on previously learned classes. A major challenge in this setting is catastrophic forgetting, where the model loses its ability to recognize old classes after learning new ones.

The problem becomes more severe when clients join or leave the system intermittently, leading to data imbalance and inconsistent learning behavior. Therefore, designing an FCIL framework that is stable, adaptive, and scalable remains a significant research challenge.

PRIVACY CHALLENGES IN FCIL

Although federated learning avoids sharing raw data, model updates such as gradients or parameters can still leak sensitive information. This risk is amplified in class-incremental

scenarios where updates may reveal details about newly introduced classes or rare samples. Hence, ensuring strong privacy guarantees is critical for real-world FCIL deployment.

Differential Privacy (DP) is widely recognized as a rigorous privacy-preserving mechanism. However, conventional DP techniques often introduce excessive noise, leading to reduced model accuracy. Moreover, most existing FCIL methods do not explicitly integrate differential privacy, leaving a gap between privacy protection and continual learning performance.

MOTIVATION

The limitations of existing FCIL frameworks—such as poor handling of evolving classes, vulnerability to catastrophic forgetting, inefficient aggregation strategies, and weak privacy protection—motivate the need for an enhanced solution. There is a strong demand for a federated learning framework that can:

- Learn new classes incrementally,
- Preserve knowledge of old classes,
- Ensure strong privacy guarantees, and
- Maintain high accuracy and fast convergence.

PROPOSED APPROACH OVERVIEW

To address these challenges, this thesis proposes an Enhanced Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework. The proposed system integrates Rényi Differential Privacy (RDP) with adaptive noise calibration to achieve tighter privacy bounds and better privacy–utility trade-offs. An attention-based dynamic aggregation mechanism is employed to intelligently weight client contributions based on model quality and data characteristics.

Furthermore, a dual-model architecture with Elastic Weight Consolidation (EWC) is introduced to preserve important parameters from previous tasks, thereby reducing catastrophic forgetting. To strengthen memory replay, a CoreSet-based memory selection strategy is used to maintain a compact yet representative set of samples.

OBJECTIVES OF THE THESIS

The primary objectives of this research are:

- To design a privacy-preserving federated class-incremental learning framework.
- To reduce catastrophic forgetting during incremental learning.
- To improve global model accuracy and convergence speed.
- To ensure strong privacy protection using advanced differential privacy techniques.

- To evaluate the effectiveness of the proposed framework in dynamic learning environments

PROBLEM STATEMENT

Despite the advantages of federated learning, existing approaches assume static class distributions and fail to effectively handle continuously evolving data. When new classes are introduced across distributed clients, the global model suffers from catastrophic forgetting, reduced accuracy, and unstable convergence. Moreover, most federated class-incremental learning frameworks do not provide strong privacy guarantees, making them unsuitable for sensitive applications. Therefore, there is a need for a robust, privacy-preserving FCIL framework that ensures continual learning, protects client data, and maintains high performance.

RESEARCH GAPS IDENTIFIED

Based on the literature survey, the following research gaps are identified:

- Limited support for dynamic and real-time class additions in federated settings
- Inadequate mechanisms to reduce catastrophic forgetting
- Weak integration of differential privacy in FCIL systems
- Static or heuristic-based aggregation strategies
- Poor balance between privacy protection and model accuracy

These gaps motivate the proposed enhanced PP-FCIL framework.

SCOPE OF THE WORK

The scope of this research includes:

- Designing a privacy-preserving federated learning framework for class-incremental scenarios
- Integrating advanced differential privacy techniques to protect client updates
- Improving aggregation strategies to handle heterogeneous and non-IID data
- Reducing catastrophic forgetting through memory consolidation techniques
- Evaluating performance using accuracy, convergence speed, privacy loss, and forgetting metrics

The work focuses on supervised learning tasks and does not address reinforcement learning or fully unsupervised settings.

SIGNIFICANCE OF THE STUDY

This research contributes to the advancement of federated learning by addressing key challenges in privacy and continual learning. The proposed framework is significant for applications involving sensitive and evolving data such as healthcare, smart cities, finance, and IoT systems. By improving privacy guarantees and learning stability, the system enhances trust, scalability, and real-world applicability of federated learning solutions.

APPLICATIONS OF THE PROPOSED SYSTEM

The proposed PP-FCIL framework can be applied in various domains, including:

- Healthcare systems with continuously evolving medical data
- Smart surveillance and security systems
- Personalized recommendation systems
- Autonomous and intelligent transportation systems
- IoT and edge-based distributed learning environments

ASSUMPTIONS AND CONSTRAINTS

The proposed system is developed under the following assumptions:

- Clients possess sufficient computational resources for local training
- Communication between clients and server is secure
- Data distributions across clients are non-IID

Constraints include communication overhead, limited client memory, and scalability challenges in large federated networks.

ETHICAL AND PRIVACY CONSIDERATIONS

This research emphasizes ethical AI development by ensuring that sensitive client data remains private. The use of Rényi Differential Privacy prevents unauthorized inference attacks and ensures compliance with data protection regulations. The framework supports responsible AI deployment in privacy-critical environments.

SUMMARY OF CHAPTER

This chapter introduced the background, motivation, challenges, and objectives of privacy-preserving federated class-incremental learning. It highlighted the limitations of existing approaches and justified the need for an enhanced PP-FCIL framework.

CHAPTER 2

LITERATURE SURVEY

2.1 MACHINE LEARNING

Machine Learning is a subfield of Artificial Intelligence that focuses on the development of algorithms and statistical models that enable computer systems to learn from data and improve their performance over time without explicit programming. A machine learning model identifies hidden patterns and relationships within data and utilizes them to make accurate predictions or decisions on unseen data.

MACHINE LEARNING ALGORITHMS

LINEAR REGRESSION

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. The model assumes a linear relationship and attempts to fit a straight line that minimizes the sum of squared errors between predicted and actual values.

Mathematically, the hypothesis function is represented as a linear combination of input features and corresponding weights. The parameters are optimized using methods such as Ordinary Least Squares or Gradient Descent. Linear regression is computationally efficient and easy to interpret, making it suitable for baseline predictive modeling.

Applications: Sales forecasting, economic analysis, weather prediction

Limitations: Poor performance with non-linear data, sensitive to outliers

LOGISTIC REGRESSION

Logistic Regression is a supervised classification algorithm that predicts the probability of a binary outcome using a logistic (sigmoid) function. Unlike linear regression, its output is constrained between 0 and 1, making it suitable for classification tasks.

The model uses maximum likelihood estimation to determine optimal parameters. Logistic regression is widely adopted due to its interpretability and statistical foundation.

Applications: Disease diagnosis, fraud detection, spam classification

Limitations: Assumes linear decision boundary, not suitable for complex relationships.

DECISION TREE

Decision Tree is a tree-structured supervised learning algorithm used for both classification and regression tasks. The model splits the dataset into subsets based on feature values using metrics such as Gini Index or Information Gain.

Decision trees are intuitive and easy to visualize. However, deep trees tend to overfit the training data if pruning is not applied.

Applications: Credit risk assessment, customer segmentation

Limitations: Overfitting, instability with small data changes

RANDOM FOREST

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy. Each tree is trained on a random subset of data and features, ensuring diversity among trees.

The final output is obtained through majority voting (classification) or averaging (regression). Random forests significantly reduce overfitting compared to individual decision trees.

Applications: Medical diagnosis, intrusion detection, bioinformatics

Limitations: Less interpretable, higher computational cost.

SUPPORT VECTOR MACHINE (SVM)

Support Vector Machine is a powerful supervised learning algorithm that finds the optimal hyperplane separating data points of different classes. The objective is to maximize the margin between classes.

SVM supports kernel functions to handle non-linear data. It is effective in high-dimensional spaces and performs well with limited training data.

Applications: Text classification, image recognition, bioinformatics

Limitations: High memory usage, sensitive to kernel choice.

NAÏVE BAYES

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes conditional independence among features, which simplifies computation.

Despite its simplifying assumption, Naïve Bayes performs well in many real-world scenarios, especially with high-dimensional data.

Applications: Email spam filtering, sentiment analysis

Limitations: Independence assumption rarely holds true

K-NEAREST NEIGHBORS (KNN)

KNN is a supervised, instance-based learning algorithm that classifies data points based on the majority class of their nearest neighbors. Distance metrics such as Euclidean or Manhattan distance are used.

KNN does not require a training phase, but prediction can be computationally expensive for large datasets.

Applications: Recommendation systems, pattern recognition

Limitations: Slow prediction, sensitive to noise and scaling

K-MEANS CLUSTERING

K-Means is an unsupervised learning algorithm used for clustering data into k distinct groups. It iteratively assigns data points to the nearest centroid and updates centroids until convergence.

The algorithm is simple and efficient but requires the number of clusters to be predefined.

Applications: Customer segmentation, image compression

Limitations: Sensitive to initial centroids, assumes spherical clusters

Hierarchical Clustering

Hierarchical Clustering builds a hierarchy of clusters using either agglomerative (bottom-up) or divisive (top-down) strategies. The clustering process is visualized using a dendrogram.

This algorithm is useful when the number of clusters is unknown and provides better interpretability.

Applications: Gene analysis, document clustering

Limitations: High computational complexity

PRINCIPAL COMPONENT ANALYSIS (PCA)

PCA is an unsupervised dimensionality reduction technique that transforms correlated variables into a set of uncorrelated principal components.

By retaining only the most significant components, PCA reduces dimensionality while preserving maximum variance.

Applications: Image processing, data visualization

Limitations: Loss of interpretability, linear assumption

ARTIFICIAL NEURAL NETWORK (ANN)

ANNs consist of interconnected neurons organized into input, hidden, and output layers. Each neuron applies a weighted sum followed by an activation function.

ANNs are capable of learning complex non-linear patterns and adapting to large datasets.

Applications: Medical diagnosis, predictive analytics

Limitations: Requires large data, difficult to interpret

CONVOLUTIONAL NEURAL NETWORK (CNN)

CNNs are specialized neural networks designed for processing spatial data such as images. They use convolutional layers to automatically extract features.

CNNs reduce manual feature extraction and achieve high accuracy in vision-related tasks.

Applications: Face recognition medical imaging

Limitations: High computational requirements

RECURRENT NEURAL NETWORK (RNN)

RNNs are designed for sequential data by incorporating feedback connections that allow information persistence.

They are suitable for tasks involving time dependencies but suffer from vanishing gradient problems.

Applications: Speech recognition, time-series forecasting

Limitations: Short-term memory issues

LONG SHORT-TERM MEMORY (LSTM)

Long Short-Term Memory (LSTM) is an advanced variant of Recurrent Neural Networks (RNNs) designed to effectively model long-range dependencies in sequential data. Traditional RNNs suffer from the vanishing and exploding gradient problems during backpropagation through time, which limits their ability to learn dependencies across long sequences. LSTM addresses this limitation by introducing a memory cell structure along with gating mechanisms that regulate information flow.

An LSTM network consists of three primary gates: the input gate, forget gate, and output gate. The forget gate determines which information from the previous cell state should be discarded, while the input gate controls the amount of new information to be added to the cell state. The output gate regulates the information passed to the next hidden state. These gates are implemented using sigmoid activation functions, enabling selective memory retention.

Applications: Stock price prediction, sentiment analysis

Limitations: High training time and complexity

GRADIENT BOOSTING (XGBOOST)

Gradient Boosting is an ensemble learning technique that builds predictive models in a sequential manner, where each new model attempts to correct the errors made by previously trained models. Unlike bagging-based ensemble methods, gradient boosting focuses on minimizing a predefined loss function by iteratively adding weak learners, typically decision trees.

XGBoost (Extreme Gradient Boosting) is an optimized and scalable implementation of gradient boosting that incorporates advanced system and algorithmic enhancements. It introduces regularization techniques to control model complexity and prevent overfitting. Both L1 and L2 regularization terms are included in the objective function, which improves generalization performance.

Applications: Risk modeling predictive analytics

Limitations: Complex tuning, risk of overfitting

2.2 FEDERATED LEARNING FUNDAMENTALS

McMahan et al. [1] introduced Federated Learning (FL) as a decentralized training framework that enables collaborative model learning without transferring raw data to a central server. Their FedAvg algorithm significantly reduced communication overhead by aggregating local model updates instead of data. This work laid the foundation for privacy-aware distributed learning but assumed static data distributions and fixed class labels.

Mohammad and Sorour [2] extended FL to mobile edge environments by proposing adaptive task allocation for asynchronous clients. Their approach improved training efficiency under heterogeneous client availability. However, the framework did not address incremental learning or catastrophic forgetting when new data classes emerge dynamically.

2.3 Federated Learning with Intermittent and Heterogeneous Clients

Díaz and García [3] studied the scalability and performance of FL in medical imaging systems with intermittent client participation. Their results showed that frequent client dropouts negatively affect convergence and accuracy. While effective for medical imaging, the approach did not consider evolving class distributions or privacy-enhancing mechanisms.

Shen et al. [4] proposed a federated learning framework for histopathology image analysis using GAN-based stain normalization. Their system improved cross-client consistency and model accuracy. However, the method focused on data normalization rather than class-incremental learning or privacy guarantees.

2.4 DECENTRALIZED AND EDGE-BASED FEDERATED LEARNING

Zhou et al. [5] proposed a decentralized peer-to-peer federated learning architecture for mobile robotic systems. Their approach improved system robustness and fault tolerance. Despite enhanced resilience, the framework lacked mechanisms to handle class-incremental learning and privacy leakage.

Huang et al. [6] introduced Fed-HANet for federated visual grasping in human–robot interaction scenarios. Their work demonstrated the feasibility of FL in robotics but did not address catastrophic forgetting or differential privacy.

Wang et al. [7] presented ATPFL for automatic trajectory prediction under a federated framework. The model dynamically adapted to user behavior, but privacy protection was limited and incremental class learning was not explored.

Du et al. [8] developed a personalized federated learning framework for driver behavior prediction. While personalization improved accuracy, the system did not provide explicit privacy guarantees or support class-incremental scenarios.

2.5 ASYNCHRONOUS AND DISTRIBUTED OPTIMIZATION IN FL

Agarwal and Duchi [9] analyzed delayed stochastic optimization in distributed systems, highlighting the impact of communication delays on convergence. Their findings influenced later asynchronous FL designs.

Lian et al. [10] proposed asynchronous parallel stochastic gradient descent for non-convex optimization, enabling faster training in distributed environments. However, the approach assumed centralized control and static learning objectives.

Further, Lian et al. [11] compared decentralized and centralized optimization, showing that decentralized methods can outperform centralized ones under certain conditions. Nevertheless, class-incremental learning and privacy aspects were not considered.

2.6 FEDERATED OPTIMIZATION UNDER SYSTEM HETEROGENEITY

Li et al. [12] investigated federated optimization in heterogeneous networks and introduced methods to mitigate client drift caused by non-IID data. Although effective in stabilizing training, their work did not handle dynamic class evolution.

Wang et al. [13] proposed adaptive federated learning for resource-constrained edge systems. Their adaptive control improved efficiency but lacked mechanisms for privacy protection and continual learning.

2.7 PRIVACY AND SECURITY IN FEDERATED LEARNING

Shokri and Shmatikov [14] were among the first to introduce privacy-preserving deep learning using differential privacy techniques. Their work demonstrated strong privacy guarantees but suffered from reduced model accuracy due to excessive noise.

Wang et al. [15] showed that federated learning is vulnerable to user-level privacy leakage, even without sharing raw data. This work highlighted the need for stronger privacy mechanisms beyond basic FL.

Ma et al. [16] analyzed security and privacy challenges in federated learning, identifying threats such as inference attacks and malicious clients.

Mothukuri et al. [17] provided a comprehensive survey on security and privacy in federated learning, categorizing attacks and defenses. However, their survey revealed a lack of privacy-preserving solutions for class-incremental learning.

Zhang et al. [18] further surveyed trustworthy federated learning, focusing on robustness, security, and privacy. The authors emphasized that continual and incremental learning remains an open challenge.

2.8 PRIVACY ATTACKS ON MACHINE LEARNING MODELS

Wang and Gong [19] demonstrated hyperparameter stealing attacks, showing that sensitive training information can be inferred from model updates. This raised concerns about model transparency in FL.

Ganju et al. [20] proposed property inference attacks that extract sensitive attributes from trained models. Their work confirmed that privacy risks persist even in decentralized learning settings.

2.9 SUMMARY AND RESEARCH GAP

- Existing FL frameworks assume static class distributions

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

- Catastrophic forgetting is largely unaddressed in federated environments
- Privacy mechanisms are often weak or reduce model utility

These limitations motivate the need for an enhanced Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework that ensures strong privacy guarantees, reduces forgetting, and maintains high accuracy in dynamic, real-world environments.

TABLE 2.1: SUMMERY OF LITERATURE SURVEY

Ref. No.	Author & Year	Focus Area	Key Contribution	Limitations / Research Gap
[1]	McMahan et al., 2017	Federated Learning	Introduced FL and FedAvg for decentralized training	Assumes static data classes; no incremental learning
[2]	Mohammad & Sorour, 2019	Asynchronous FL	Adaptive task allocation for mobile edge FL	No support for class-incremental learning or privacy
[3]	Díaz & García, 2023	Medical Imaging FL	Studied scalability with intermittent clients	Lacks incremental learning and strong privacy
[4]	Shen et al., 2023	Medical Image FL	GAN-based stain normalization in FL	No catastrophic forgetting mitigation
[5]	Zhou et al., 2023	Decentralized FL	P2P FL for resilient robotic systems	No privacy-preserving class learning
[6]	Huang et al., 2023	Robotics FL	FL for visual grasping tasks	Ignores privacy leakage and forgetting
[7]	Wang et al., 2022	Prediction Models	Automatic trajectory prediction in FL	Static learning; no incremental class handling
[8]	Du et al., 2023	Personalized FL	Driver behavior prediction using FL	Limited privacy guarantees

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

[9]	Agarwal & Duchi, 2012	Distributed Optimization	Delayed stochastic optimization	Not designed for FL or privacy
[10]	Lian et al., 2015	Asynchronous SGD	Parallel SGD for non-convex problems	No FL or privacy consideration
[11]	Lian et al., 2017	Decentralized Learning	Compared decentralized vs centralized SGD	No incremental or privacy focus
[12]	Li et al., 2020	Heterogeneous FL	Addressed client drift in non-IID data	No class-incremental learning
[13]	Wang et al., 2019	Edge-based FL	Adaptive FL for resource-constrained systems	Privacy not guaranteed
[14]	Shokri & Shmatikov, 2015	Privacy-Preserving DL	Introduced differential privacy in DL	High noise reduces accuracy
[15]	Wang et al., 2019	Privacy Attacks	Identified user-level privacy leakage in FL	No defense for FCIL
[16]	Ma et al., 2020	FL Security	Analyzed FL security threats	No incremental learning solution
[17]	Mothukuri et al., 2021	FL Survey	Survey on FL privacy and security	Highlights lack of FCIL solutions
[18]	Zhang et al., 2023	Trustworthy FL	Survey on robustness and privacy	Incremental learning still open
[19]	Wang & Gong, 2018	Model Attacks	Hyperparameter stealing attacks	Emphasizes privacy risks
[20]	Ganju et al., 2018	Inference Attacks	Property inference attacks on models	Shows need for strong DP mechanisms

CHAPTER 3

SYSTEM ANALYSIS

3.1 INTRODUCTION

System analysis is a critical phase in the design of any intelligent learning framework, as it identifies existing limitations and defines system requirements for the proposed solution. In the context of federated class-incremental learning, system analysis focuses on challenges related to data privacy, evolving class distributions, catastrophic forgetting, and aggregation efficiency. This chapter presents a detailed analysis of the existing system, problem formulation, proposed system overview, functional and non-functional requirements, and feasibility analysis.

3.2 EXISTING SYSTEM ANALYSIS

Traditional federated learning frameworks enable decentralized model training without sharing raw data. However, most existing systems assume static data classes and predefined training tasks. When new classes are introduced incrementally across distributed clients, the global model tends to forget previously learned knowledge, leading to catastrophic forgetting. Additionally, existing aggregation strategies are largely static and do not consider client heterogeneity, data imbalance, or temporal consistency.

From a privacy perspective, although federated learning avoids direct data sharing, model updates can still leak sensitive information. Most existing FCIL approaches do not integrate strong differential privacy mechanisms, making them vulnerable to inference attacks. These limitations highlight the need for an enhanced privacy-preserving and adaptive federated learning framework.

3.3 PROBLEM STATEMENT

The major problem addressed in this research is the lack of an effective federated learning framework that can handle class-incremental data while ensuring strong privacy protection. Existing approaches fail to balance learning new classes, preserving old knowledge, and protecting client data. Therefore, a system is required that supports continuous learning, minimizes catastrophic forgetting, and provides robust privacy guarantees in distributed environments.

3.4 PROPOSED SYSTEM OVERVIEW

The proposed system introduces an Enhanced Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework. It integrates Rényi Differential Privacy (RDP) with adaptive noise calibration to ensure stronger privacy guarantees. An attention-based dynamic aggregation

mechanism is employed to intelligently weight client updates based on model quality and data characteristics. To preserve old knowledge while learning new classes, a dual-model architecture enhanced with Elastic Weight Consolidation (EWC) is adopted. Additionally, CoreSet-based memory selection is used to maintain a representative subset of past data, reducing catastrophic forgetting.

3.5 SYSTEM ARCHITECTURE DESCRIPTION

The system consists of multiple federated clients, a central server, and a secure communication channel. Each client trains a local model on private data and applies RDP-based noise before sharing updates. The server aggregates these updates using attention-based weighting and updates the global model. Memory replay and EWC ensure stability during incremental learning. The updated global model is then distributed back to clients for further training.

3.6 FUNCTIONAL REQUIREMENTS

The functional requirements of the proposed system include:

- Support federated learning without sharing raw client data
- Enable class-incremental learning across distributed clients
- Apply differential privacy to protect model updates
- Perform adaptive and intelligent aggregation of client models
- Preserve previously learned knowledge using EWC
- Maintain representative memory using CoreSet selection
- Evaluate model performance after each learning round

3.7 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements include:

- **Privacy:** Strong protection against data and model inference attacks
- **Scalability:** Ability to handle an increasing number of clients
- **Efficiency:** Reduced communication and faster convergence
- **Reliability:** Stable learning under intermittent client participation
- **Security:** Secure transmission of model updates

3.8 FEASIBILITY ANALYSIS

3.8.1 Technical & Economic Feasibility:

The system is supported by existing federated learning and deep learning frameworks, while

decentralized computation reduces infrastructure costs and eliminates the need for additional hardware.

3.8.2 Operational Feasibility:

The framework can be smoothly deployed in real-world applications such as healthcare and IoT with minimal changes to existing workflows.

3.9 ADVANTAGES OF THE PROPOSED SYSTEM

- Strong privacy protection using Rényi Differential Privacy
- Reduced catastrophic forgetting through EWC and memory replay
- Improved accuracy and convergence speed
- Adaptive aggregation suited for heterogeneous clients

3.10 USE CASE ANALYSIS

Use case analysis identifies the interactions between system users (clients and server) and the proposed PP-FCIL framework. It helps in understanding how data flows and how each module behaves during incremental learning and aggregation.

Actors:

- Federated Client
- Federated Server

Use Cases:

- Local Model Training
- Privacy-Preserving Update Generation
- Secure Model Aggregation
- Incremental Class Learning
- Global Model Distribution

3.11 DATA FLOW ANALYSIS

Data flow analysis describes how information moves through the system:

- Local client data is used only for on-device training.
- Model gradients are processed through the RDP mechanism.

- Privacy-protected updates are sent to the server.
- The server aggregates updates and updates the global model.
- The global model is redistributed to clients.

This ensures data privacy and secure communication throughout the learning process.

3.12 CONSTRAINTS OF THE SYSTEM

The proposed system operates under certain constraints:

- Limited client computation and memory resources
- Communication delays between clients and server
- Non-IID data distribution across clients
- Limited memory size for exemplar storage
- Trade-off between privacy noise and accuracy

3.13 RISK ANALYSIS

Potential risks involved in the system include:

- Excessive noise addition affecting model accuracy
- Client dropout during training rounds
- High communication overhead in large networks
- Memory overflow if class growth is rapid

These risks are mitigated through adaptive noise calibration, dynamic aggregation, and efficient memory selection.

3.14 PERFORMANCE METRICS CONSIDERED

The system performance is evaluated using the following metrics:

- Incremental Accuracy
- Privacy Loss (ϵ value)
- Forgetting Rate

- Convergence Speed
- Communication Cost

These metrics provide a holistic evaluation of the proposed framework.

3.15 DESIGN GOALS

The primary design goals of the proposed PP-FCIL system are:

- Preserve user data privacy at all stages
- Support continuous and scalable learning
- Minimize catastrophic forgetting
- Maintain high model accuracy
- Ensure efficient communication and aggregation

3.16 COMPARISON WITH EXISTING SYSTEM

Aspect	Existing System	Proposed System
Data Sharing	Model updates only	Model updates with RDP
Class Handling	Static classes	Dynamic class-incremental
Privacy	Limited	Strong (RDP-based)
Aggregation	Static weighting	Attention-based dynamic
Forgetting	High	Reduced using EWC + CoreSet

TABLE :3.1 COMPARSION WITH EXISTING SYSTEM

3.17 ASSUMPTIONS MADE

The following assumptions are considered during system analysis:

- Clients act honestly during training
- Secure communication channels exist
- Sufficient labeled data is available at clients
- Model architectures are compatible across clients

3.18 VALIDATION STRATEGY

The system is validated through:

- Incremental learning experiments
- Privacy leakage analysis
- Comparison with baseline FL methods
- Statistical evaluation of performance improvements

3.19 SUMMARY OF THE CHAPTER

This chapter presented a comprehensive system analysis of the proposed PP-FCIL framework. It discussed the limitations of existing systems, defined the problem statement, and described the proposed architecture and requirements. The analysis confirms that the proposed system is feasible, efficient, and capable of addressing privacy and continual learning challenges in federated environments.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

System design defines how the proposed solution is structured, how components interact, and how system objectives are achieved in practice. Based on the system analysis presented in Chapter 3, this chapter elaborates the design of the Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework. The design focuses on privacy preservation, adaptability to evolving classes, robustness against catastrophic forgetting, and efficient aggregation across heterogeneous clients. Detailed architectural components, module-level descriptions, UML-based representations, and design considerations are discussed to provide a complete blueprint for implementation.

4.2 OVERALL SYSTEM ARCHITECTURE

The proposed PP-FCIL system adopts a **client–server federated learning architecture**, where multiple distributed clients collaborate to train a shared global model under the coordination of a central server. Unlike traditional centralized systems, raw data never leaves client devices. Instead, each client performs local training and shares only privacy-protected model updates.

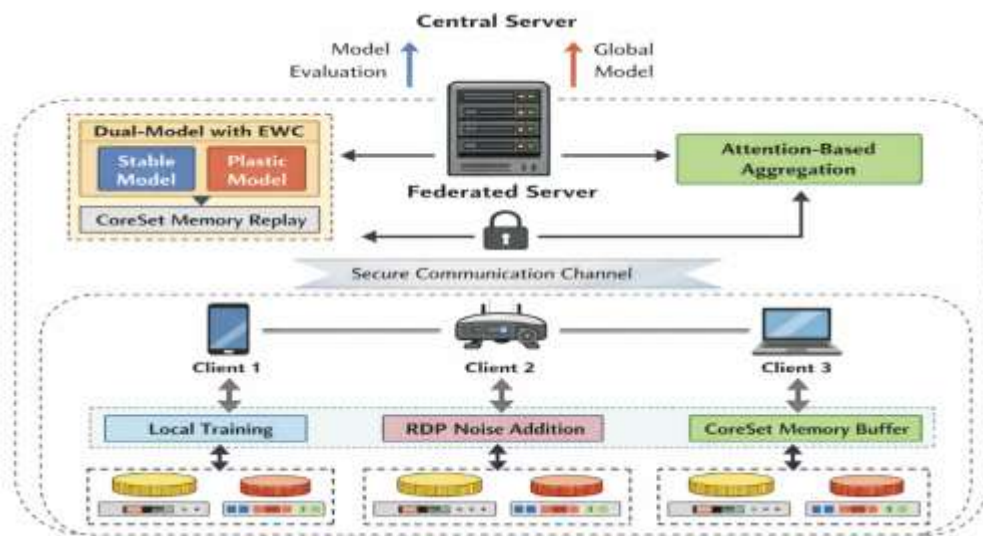


Figure : 4.1 client–server federated learning architecture

The architecture integrates:

- Differential privacy at the client side,
- Adaptive aggregation at the server,
- Dual-model consolidation to handle old and new knowledge,
- Memory replay to stabilize incremental learning.

This layered architecture ensures scalability, privacy, and learning stability even when new classes appear dynamically across clients.

The proposed architecture illustrates a privacy-preserving federated class-incremental learning framework designed to support continuous learning while ensuring strong data privacy and reduced catastrophic forgetting. The architecture is divided into client-side operations and server-side coordination, connected through a secure communication channel.

Federated Clients

Multiple federated clients (Client 1, Client 2, Client 3, ...) participate in collaborative training. Each client holds its own **private and distributed dataset**, which may contain new classes introduced over time.

- Clients perform **local model training** using their private data.
- No raw data is shared with the server, ensuring data confidentiality.
- Each client supports class-incremental learning, allowing new classes to be learned dynamically.

Local Training Module

The local training module updates the client model using locally available data.

- Training is performed independently at each client.
- Supports learning new classes while retaining previously learned classes.
- Generates gradient updates after each training round.

Rényi Differential Privacy (RDP) Noise Addition

Before transmitting model updates, each client applies **RDP-based noise addition**.

- Adaptive noise is added based on gradient sensitivity.
- Provides strong privacy guarantees with tighter ϵ bounds.
- Prevents sensitive information leakage through model updates.

This step ensures that privacy is preserved even if an adversary observes the transmitted updates.

Secure Communication Channel

A secure and encrypted communication channel connects clients and the federated server.

- Ensures confidentiality and integrity of model updates.
- Protects against interception and tampering attacks.
- Supports reliable transmission in distributed environments

Federated Server

The federated server acts as the central coordinator of the system.

- Receives privacy-preserved updates from all clients.
- Manages global aggregation, memory replay, and evaluation.
- Distributes the updated global model back to clients.

Attention-Based Dynamic Aggregation

Instead of simple averaging, the server uses **attention-based aggregation**.

- Dynamically assigns weights to client updates.
- Considers client model quality, data distribution similarity, and update consistency.
- Improves convergence speed and overall model accuracy.

This mechanism handles heterogeneity across clients effectively.

Dual-Model Architecture with Elastic Weight Consolidation (EWC)

To mitigate catastrophic forgetting, the server maintains a dual-model structure:

- **Stable Model:** Preserves knowledge from previously learned classes.
- **Plastic Model:** Adapts quickly to new classes.

Elastic Weight Consolidation assigns importance to critical parameters, preventing significant changes to those parameters during new learning.

CoreSet Memory Replay

The CoreSet memory replay module stores a compact and representative subset of past sample.

- Selected samples capture the diversity of earlier tasks.
- Used during training to reinforce old knowledge.
- Helps reduce forgetting and improve class-incremental performance.

Model Evaluation and Global Model Update

After aggregation and consolidation:

- The global model is evaluated for accuracy and stability.
- Performance metrics are generated.

- The updated global model is distributed back to clients for the next learning round.

Overall Working Flow

1. Clients train local models on private data
2. RDP noise is added to protect updates
3. Secure updates are sent to the server
4. Server aggregates updates using attention mechanism
5. Dual-model with EWC consolidates knowledge
6. CoreSet memory replay reduces forgetting
7. Global model is evaluated and redistributed

The architecture effectively integrates **privacy preservation, adaptive aggregation, and memory consolidation** into a unified federated learning framework. It supports continuous class-incremental learning while ensuring strong privacy guarantees and improved learning stability, making it suitable for real-world distributed and sensitive data applications

4.3 MODULE-WISE SYSTEM DESIGN

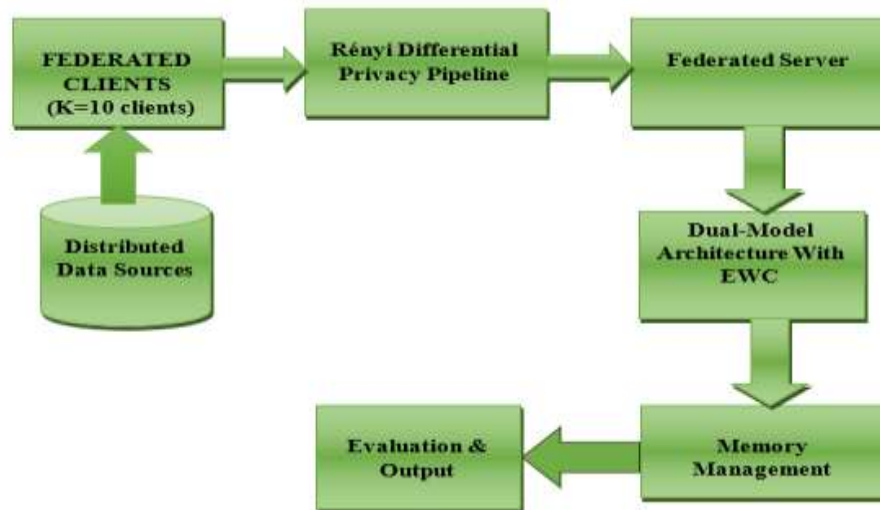


Figure: 4.3 Module-Wise System Design

4.3.1 Distributed Data Source Module

This module represents the decentralized datasets stored at individual client devices. Each client holds its own data collected from real-world environments such as sensors, medical systems, or user interactions. The data is **non-IID**, heterogeneous, and evolves over time with the introduction of new classes.

Key characteristics:

- Data remains local and private
- Supports dynamic class growth
- Reflects real-world distributed environments

This module ensures compliance with data protection regulations while enabling collaborative learning.

4.3.2 Federated Client Module

Each federated client is responsible for:

- Receiving the global model from the server,
- Training the model locally on private data,
- Handling newly introduced classes incrementally,
- Generating model updates after each training round.

Clients operate independently and may join or leave the system dynamically. The local training process is optimized to minimize computational overhead while supporting continual learning. Only model parameters or gradients are shared, ensuring data confidentiality.

4.3.3 Rényi Differential Privacy (RDP) Module

The RDP module enforces strong privacy protection by adding carefully calibrated noise to local gradients before transmission. Unlike traditional differential privacy, Rényi Differential Privacy provides **tighter privacy accounting** and improved privacy–utility trade-offs.

Design features:

- Adaptive noise calibration based on gradient sensitivity
- Dynamic adjustment according to learning phase
- Protection against inference and reconstruction attacks

This module ensures that sensitive client information cannot be inferred from shared updates.

4.3.4 Attention-Based Dynamic Aggregation Module

The aggregation module is a critical component of the server. Instead of using simple averaging, this design employs an **attention-based mechanism** that dynamically learns the importance of each client update.

Aggregation factors include:

- Local model validation accuracy
- Data distribution similarity

- Temporal consistency of updates
- Gradient alignment with global objectives

This intelligent aggregation improves convergence speed, handles client heterogeneity, and enhances global model performance.

4.3.5 Dual-Model Architecture with Elastic Weight Consolidation (EWC)

To address catastrophic forgetting, the system maintains a **dual-model structure**:

- A **plastic model** for learning new classes,
- A **stable model** for retaining old knowledge.

Elastic Weight Consolidation assigns higher importance to parameters that are critical for previously learned tasks. During training, changes to these parameters are penalized, ensuring that old knowledge is preserved while learning new information.

This design balances **stability and plasticity**, a key requirement for class-incremental learning.

4.3.6 Memory Management Module (CoreSet Selection)

The memory management module maintains a limited-size memory buffer that stores representative samples from previous tasks. Instead of random selection or herding, **CoreSet-based selection** is used.

Advantages:

- Preserves feature space diversity
- Minimizes memory usage
- Improves replay effectiveness

Stored samples are replayed during training to reinforce old knowledge and reduce forgetting.

4.3.7 Global Model Update and Distribution Module

After aggregation and consolidation, the updated global model is evaluated and redistributed to all participating clients. This ensures synchronization across the network and enables continuous learning across federated rounds.

This module ensures:

- Consistent global updates
- Iterative performance improvement
- Smooth integration of new classes

4.4 UML DIAGRAM DESIGN

4.4.1 Use Case Diagram

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

The use case diagram captures the functional interactions between federated clients and the server. Major use cases include local training, privacy enforcement, secure aggregation, memory replay, and global model distribution. The Use Case Diagram illustrates the functional interactions between the federated clients and the federated server within the Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) system. Federated clients act as primary users of the system, performing local model training on private datasets while supporting the introduction of new classes over time. The federated server serves as the central coordinator, managing global aggregation, privacy enforcement, and model updates. Key use cases include local training, class-incremental learning, privacy-preserving update generation, secure transmission of updates, adaptive model aggregation, memory replay, and global model distribution.

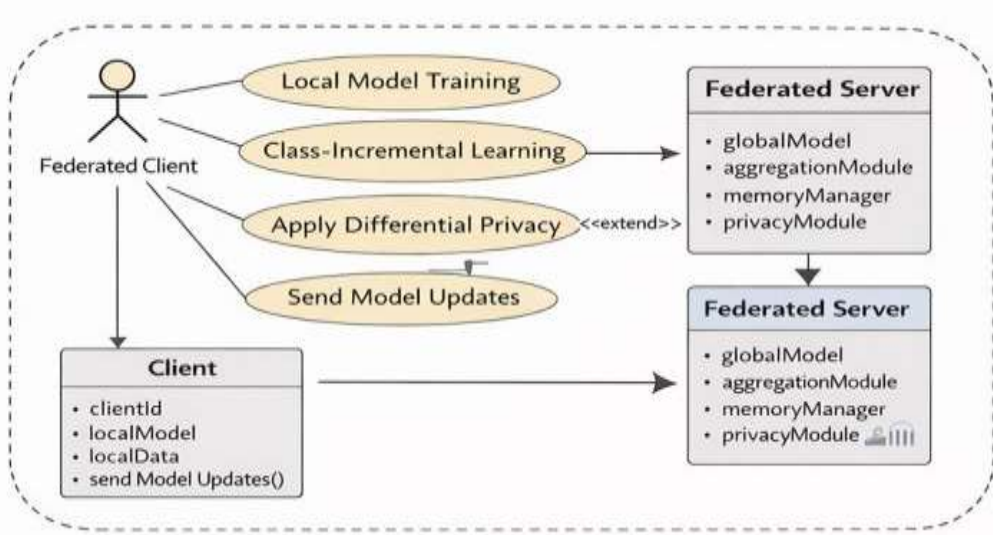


Figure : 4.4.1 Use Case Diagrams

4.4.2 Class Diagram

The class diagram defines structural relationships between system components such as Client, Server, GlobalModel, PrivacyModule, AggregationModule, and MemoryManager. It illustrates data attributes, methods, and associations. The Class Diagram represents the static structure of the PP-FCIL system by defining major classes, their attributes, methods, and relationships. Core classes include Client, FederatedServer, GlobalModel, PrivacyModule (RDP), AggregationModule, EWCMModel, and MemoryManager (CoreSet). The Client class is responsible for local training and update generation, while the FederatedServer manages aggregation and global coordination. The PrivacyModule ensures differential privacy before updates are transmitted, and the AggregationModule dynamically combines client updates using attention mechanisms. The

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

MemoryManager supports exemplar storage and replay to reduce forgetting. This diagram clearly depicts modular design, promoting scalability and maintainability of the system.

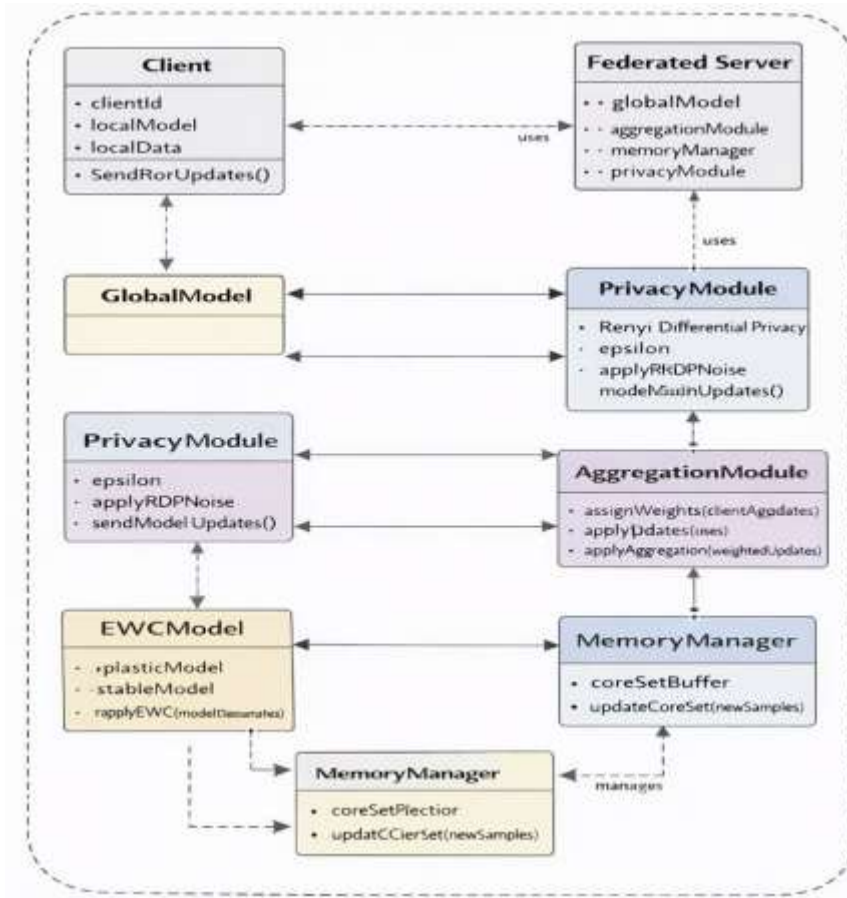


Figure: 4.4.2 : Class Diagram

4.4.3 Sequence Diagram

The sequence diagram illustrates the chronological flow of operations from client-side training to server-side aggregation and redistribution. It highlights the interaction order and dependency among system components. The Sequence Diagram illustrates the time-ordered interactions between clients and the server during a federated learning round. The process begins when the federated server distributes the current global model to all participating clients. Each client then performs local training on its private data and incorporates new classes incrementally. Before transmitting updates, the client applies Rényi Differential Privacy to protect sensitive information. The privacy-preserved updates are securely sent to the server, where attention-based dynamic aggregation is performed. The aggregated model is then consolidated using a dual-model

architecture with Elastic Weight Consolidation. Finally, the updated global model is evaluated and redistributed to clients, completing one learning cycle.

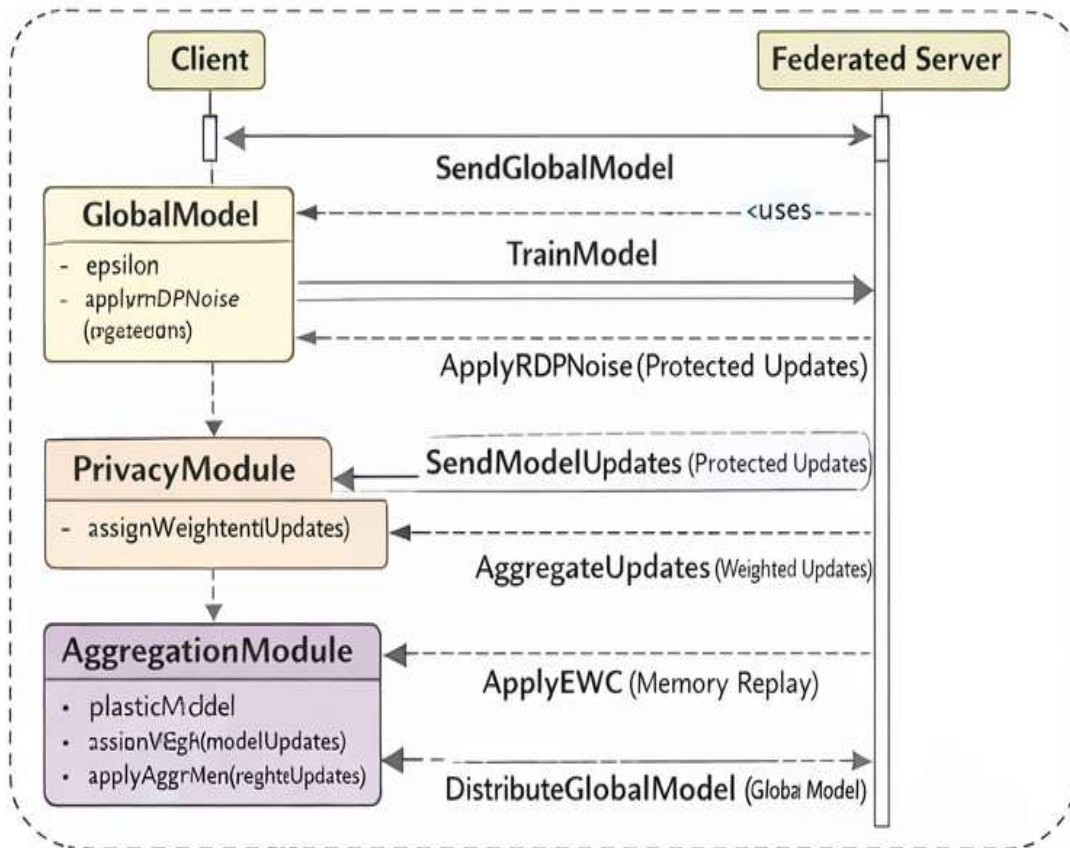


Figure: 4.4.3 : Sequence Diagram

4.4.4 Activity Diagram

The activity diagram represents the workflow of the PP-FCIL system, including initialization, training, privacy application, aggregation, memory replay, and evaluation phases. The Activity Diagram presents the workflow of the PP-FCIL system, emphasizing the sequence of operations and decision points. The process starts with global model initialization, followed by model distribution to federated clients. Clients perform local training and apply differential privacy mechanisms before sending updates. The server receives these updates and performs adaptive aggregation. Memory replay and EWC-based consolidation are then applied to reduce catastrophic forgetting. The system evaluates the updated model and decides whether further learning rounds are required. This diagram effectively captures the continuous and iterative nature of federated class-incremental learning.

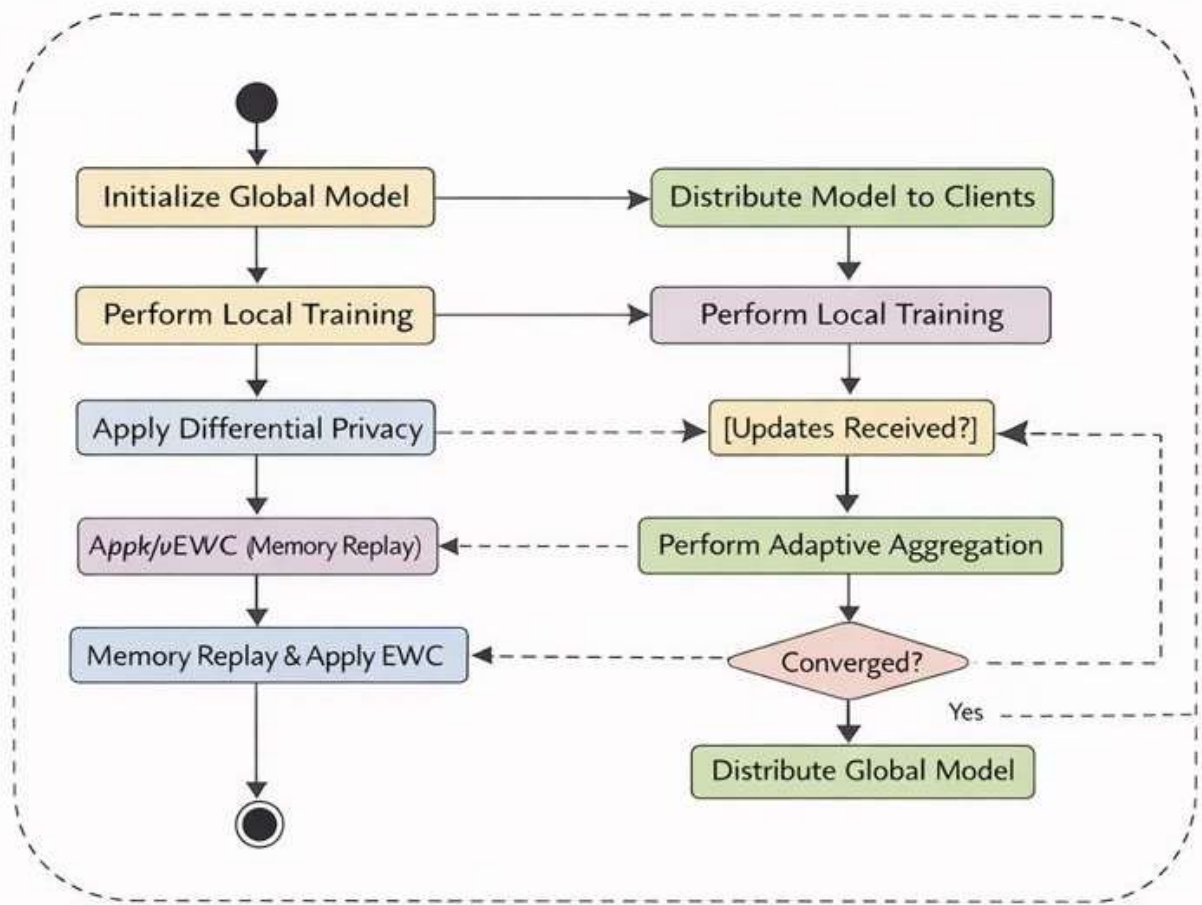


Figure : 4.4.4 : Activity Diagram

4.5 DATA FLOW DESIGN

The data flow design ensures:

- No raw data transmission
- Secure model update sharing
- Controlled information flow through privacy modules
- Centralized aggregation and evaluation

This design minimizes attack surfaces and preserves system integrity.

4.6 SECURITY AND PRIVACY DESIGN

Security is enforced at multiple levels:

- Differential privacy for model updates
- Secure communication protocols

- Robust aggregation against malicious or low-quality updates

The system design prioritizes trust, confidentiality, and resilience against attacks.

4.7 DESIGN CONSTRAINTS AND TRADE-OFFS

Key constraints include:

- Client resource limitations
- Communication bandwidth
- Privacy–accuracy trade-off
- Memory size restrictions

Adaptive mechanisms are incorporated to balance these constraints effectively.

4.8 DESIGN OBJECTIVES ACHIEVED

The proposed design achieves:

- Strong privacy protection
- Reduced catastrophic forgetting
- Improved convergence speed
- Scalability to large client populations
- Practical applicability in real-world systems

4.9 INPUT AND OUTPUT DESIGN

Input and Output Design defines how data enters the system, how it is processed, and how results are presented. In the proposed **Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL)** framework, careful input and output design is essential to ensure privacy, efficiency, and scalability while supporting continuous learning.

Input Design

The input design focuses on securely accepting data and model updates from distributed clients without exposing sensitive information.

Client-Side Input

- Local datasets stored at each federated client act as primary inputs.
- Data may include new classes introduced incrementally over time.
- Input data is heterogeneous and non-IID across clients.
- Raw data is never transmitted to the federated server.

Model Input

- The global model received from the server is an input to each client.

- This model serves as the base for local training and incremental updates.
- Model parameters are synchronized periodically across learning rounds.

Training Parameters Input

- Learning rate, batch size, number of local epochs, and privacy budget (ϵ) are provided as input parameters.
- These parameters control training behavior and privacy strength.

Privacy Parameters Input

- Rényi Differential Privacy parameters such as noise scale and sensitivity values are provided as inputs.
- These parameters are dynamically adjusted based on gradient sensitivity.

Output Design

The output design ensures meaningful and secure presentation of learning results and performance metrics.

Client-Side Output

- Privacy-protected model updates (gradients or weights) are generated as outputs.
- No raw data or sensitive features are exposed.
- Updates are encrypted before transmission.

Server-Side Output

- Aggregated global model parameters after attention-based aggregation.
- Updated global model incorporating EWC and memory replay mechanisms.

Performance Metrics Output

- Incremental classification accuracy
- Forgetting rate
- Privacy loss (ϵ value)
- Convergence time
- Communication overhead

These metrics help evaluate the effectiveness of the proposed system.

Final Output

- A stable and privacy-preserving global model capable of recognizing both old and new classes.
- Improved learning accuracy with reduced catastrophic forgetting.

Advantages of the Input and Output Design

- Ensures complete data privacy by avoiding raw data transmission.
- Supports dynamic and incremental class learning.
- Provides meaningful outputs for system evaluation and decision-making.
- Scales efficiently with an increasing number of clients.

The input and output design of the PP-FCIL framework ensures secure data handling, effective incremental learning, and clear result interpretation. By carefully managing inputs and outputs at both client and server levels, the system achieves strong privacy protection, high accuracy, and robustness in real-world federated learning environments

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

This chapter describes the practical implementation of the proposed Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework. The implementation focuses on realizing the system design presented in Chapter 4 by integrating federated learning, differential privacy, adaptive aggregation, and memory consolidation mechanisms. The chapter outlines the software environment, implementation methodology, module-wise implementation, and overall execution flow of the system.

5.2 OVERALL IMPLEMENTATION WORKFLOW

The system implementation follows an iterative federated learning cycle:

1. Initialize the global model at the federated server
2. Distribute the global model to participating clients
3. Perform local training with class-incremental data
4. Apply Rényi Differential Privacy to model updates
5. Securely transmit updates to the server
6. Perform attention-based dynamic aggregation
7. Apply EWC-based dual-model consolidation
8. Perform CoreSet-based memory replay
9. Update and redistribute the global model

This cycle continues until convergence is achieved.

5.3 MODULE-WISE IMPLEMENTATION

5.3.1 Federated Client Implementation

Each federated client performs local model training using its private dataset. The client maintains:

- Local training data
- Local model copy
- Incremental class handling logic

The client updates the model by learning new classes while preserving previously learned representations. Only model parameters are generated as output, ensuring data privacy.

5.3.2 Local Training and Class-Incremental Learning

Local training is performed using mini-batch gradient descent. When new classes appear, the model architecture is updated dynamically to accommodate new output nodes. Training includes replay of stored samples to prevent forgetting.

5.3.3 Rényi Differential Privacy Implementation

Before sharing updates, each client applies Rényi Differential Privacy (RDP) to gradients.

Implementation steps include:

- Gradient clipping to bound sensitivity
- Adaptive noise generation
- Noise injection based on privacy budget (ϵ)

This ensures strong privacy guarantees while minimizing accuracy degradation.

5.3.4 Secure Communication Implementation

Model updates are transmitted through a secure communication channel. Encryption techniques ensure confidentiality and integrity of updates exchanged between clients and the federated server.

5.3.5 Attention-Based Dynamic Aggregation Implementation

At the server side, client updates are aggregated using an attention-based weighting mechanism.

The aggregation module:

- Evaluates client update quality
- Assigns dynamic weights
- Combines weighted updates into a global model

This approach improves convergence speed and robustness under non-IID data distributions.

5.3.6 Dual-Model Architecture with EWC Implementation

The server maintains two models:

- **Plastic Model** for learning new classes
- **Stable Model** for retaining old knowledge

Elastic Weight Consolidation (EWC) assigns importance values to parameters, preventing significant changes to parameters critical for previous tasks.

5.3.7 CoreSet Memory Management Implementation

The memory manager maintains a limited buffer of representative samples using CoreSet selection.

Implementation features:

- Selection of diverse samples
- Controlled memory size
- Replay during training

This significantly reduces catastrophic forgetting in class-incremental learning.

5.4 ALGORITHM DESCRIPTION

Algorithm: PP-FCIL Training Procedure

1. Initialize global model
2. For each communication round:
 - a. Distribute global model to clients
 - b. Clients perform local training
 - c. Apply RDP to local updates
 - d. Send protected updates to server
 - e. Perform attention-based aggregation
 - f. Apply EWC and memory replay
 - g. Update global model
3. Repeat until convergence

5.5 PERFORMANCE MONITORING

During implementation, the following metrics are monitored:

- Incremental accuracy
- Forgetting rate
- Privacy loss (ϵ)
- Training time
- Communication overhead

These metrics are later analyzed in Chapter 7.

5.6 IMPLEMENTATION CHALLENGES

Key challenges encountered include:

- Balancing privacy noise and model accuracy
- Handling non-IID and imbalanced data
- Managing memory constraints
- Ensuring stable convergence

These challenges are addressed through adaptive mechanisms and optimized design choices.

5.7 IMPLEMENTATION ENVIRONMENT

The proposed system is implemented using widely adopted machine learning and federated learning tools to ensure scalability and reproducibility.

Hardware Requirements

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: 20 GB free disk space
- Optional GPU support for faster training

Software Requirements

- Programming Language: Python
- Deep Learning Framework: TensorFlow / PyTorch
- Federated Learning Framework: Flower / PySyft (conceptual support)
- Numerical Libraries: NumPy, SciPy
- Visualization Tools: Matplotlib
- Operating System: Windows / Linux

The following software tools and frameworks are used:

- Python programming language
- Reinforcement learning libraries (PyTorch)
- Simulation environment for network modeling
- Blockchain framework for secure logging
- Visualization tools for performance monitoring

The choice of these tools ensures flexibility, scalability, and ease of experimentation.

The following tools and libraries are used:

- Python 3.x
- Scikit-learn
- NLTK / SpaCy
- NumPy and Pandas
- Matplotlib for visualization

A Cloud-Based Platform for Interactive Computing, Data Science, and Machine Learning

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

The top screenshot shows a Google Colab notebook titled 'Zapier Test.ipynb'. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for commands, code, and text. A 'Variables' panel on the left lists 'date_picker' and 'today' as date types. The main code cell contains a prompt 'a calendar with date picker' and a code snippet that imports 'datetime', sets 'today' to the current date, and assigns 'date_picker' to 'today'. The output shows 'datetime.date(2025, 5, 8)'. A 'Generate' button is visible at the top right of the code cell.

The bottom screenshot shows a Google Colab notebook titled 'Music Transcription with Transformers'. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for code, text, and copy to drive. A 'Table of contents' panel on the left lists sections: 'Setup Environment', 'Imports and Definitions', 'Load Model', 'Upload Audio', 'Transcribe Audio', and 'Download MIDI Transcription'. The main content area shows a diagram of the music transcription process: 'Segmented Waveform' is converted to a 'Spectrogram', which is then processed by an 'Encoder' and a 'Decoder' to produce 'MIDI-like Output Tokens'. The output tokens are shown as a list of tokens: '<time 73.1> <vel 61> <pitch 60> <time 73.3> <vel 82> <pitch 64> <time 73.5> <vel 60> <pitch 60>'. A piano roll visualization shows the output tokens as horizontal bars on a piano roll.

Figure: 5.7.1 : Implementation Environment

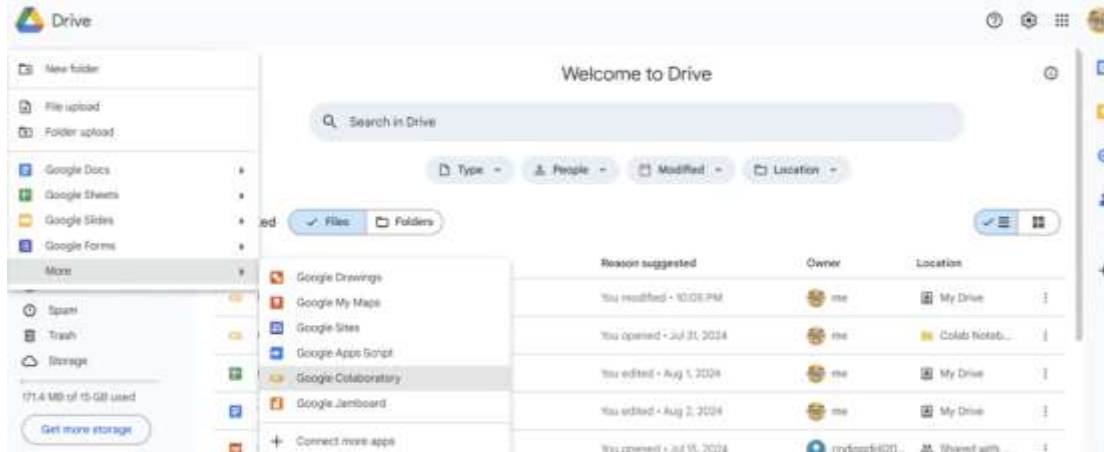


Figure : 5.7.2 : Google Drive Dashboard Interface

Introduction

In recent years, cloud computing has transformed the way software development, data analysis, and machine learning research are conducted. Traditional computing environments often require high-performance hardware, complex software installations, and continuous maintenance. These requirements can act as barriers for students, researchers, and developers, especially those with limited resources.

To overcome these challenges, Google introduced Google Colaboratory, commonly known as Google Colab. Google Colab is a cloud-based interactive development environment that allows users to write, execute, and share Python code through a web browser. It is built on top of Jupyter Notebook technology and provides free access to powerful computing resources such as Graphics Processing Units (GPUs) and Tensor Processing Units (TPUs).

Google Colab has become an essential tool in domains such as:

- Data Science
- Artificial Intelligence (AI)
- Machine Learning (ML)
- Deep Learning (DL)
- Academic research
- IoT data analysis
- Image and signal processing

Its ease of use, accessibility, and collaboration features make it particularly popular in educational institutions and research communities.

Evolution of Cloud-Based Notebook Environments

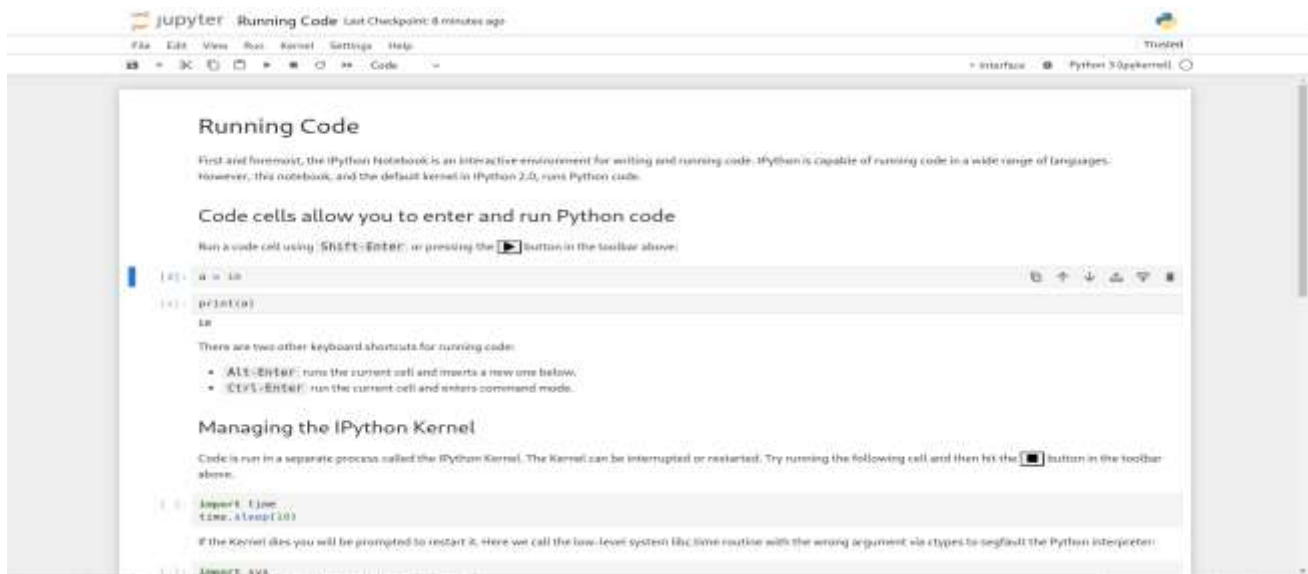


Figure : 5.7.3 : Jupyter Notebook Kernel and Code Cells View

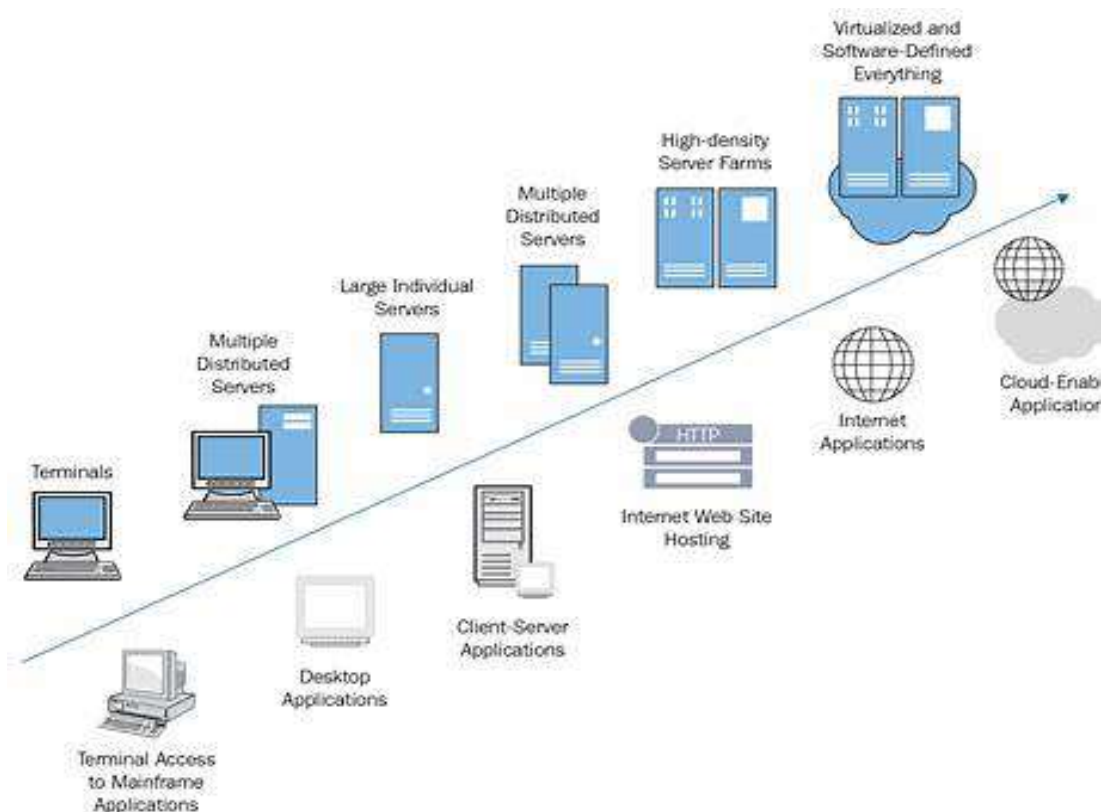


Figure: 5.7.4 : Client–Server to Cloud Computing Architecture

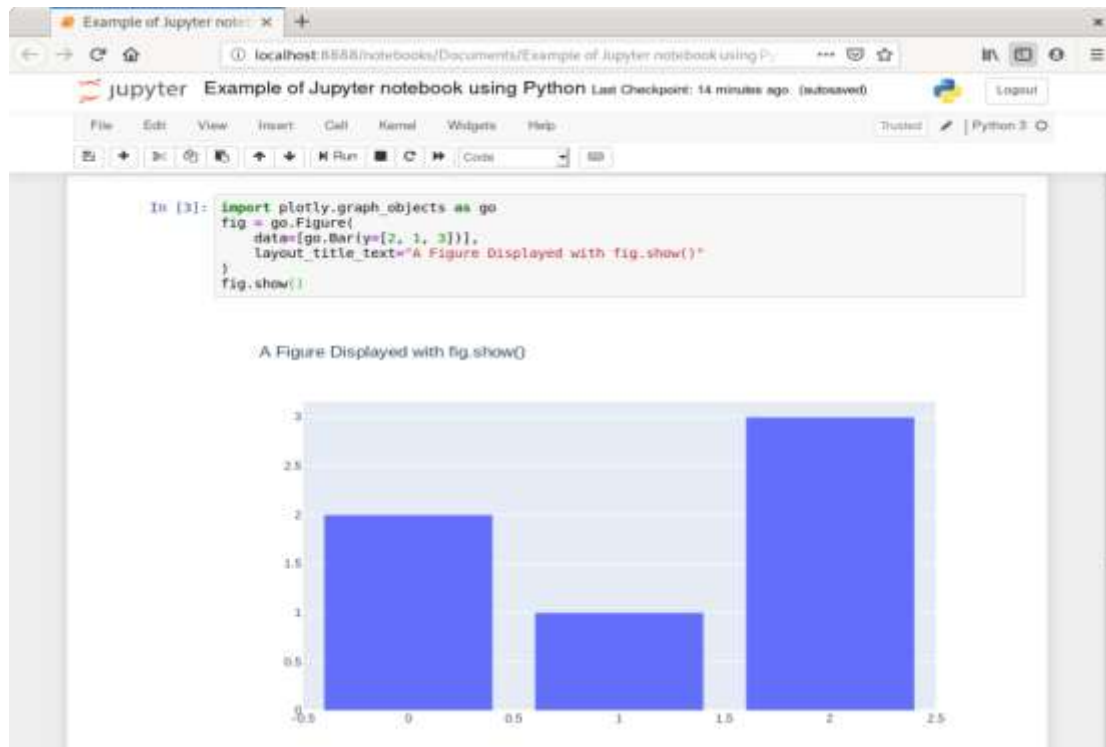


Figure: 5.7.5 : Python Data Visualization in Jupyter Notebook

Traditional Local Development

Before cloud notebooks, developers relied heavily on:

- Local PCs or laptops
- Installed IDEs (MATLAB, Python IDLE, PyCharm, etc.)
- Manually configured libraries and dependencies

This approach suffered from several drawbacks:

- Hardware limitations
- Software compatibility issues
- Time-consuming setup
- Difficulty in collaboration

Emergence of Jupyter Notebooks

Jupyter Notebooks introduced an interactive computing paradigm by allowing:

- Code execution in cells
- Inline visualization
- Markdown-based documentation

However, Jupyter notebooks still required local installation and sufficient system resources.

Introduction of Google Colab

Google Colab extended Jupyter Notebook functionality by hosting notebooks entirely in the cloud. This eliminated installation requirements and enabled:

- Browser-based execution
- Real-time collaboration
- Scalable computing power

What is Google Colab?

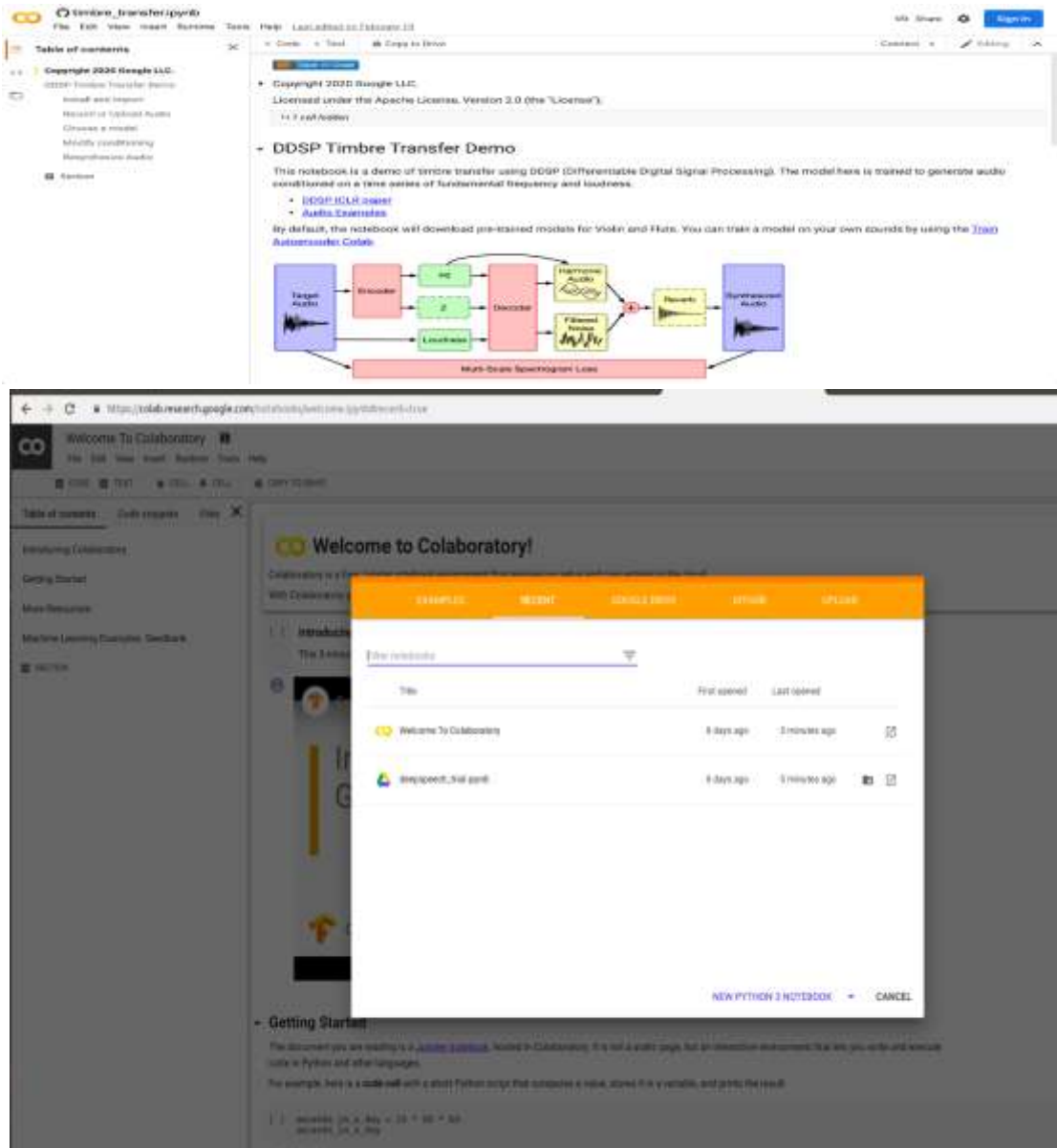


Figure: 5.7.6 : Google Colab Environment for DDSP Audio Processing

Google Colab is a **free cloud service** that enables users to:

- Write and execute Python code online
- Develop ML/DL models
- Analyze large datasets
- Share notebooks easily

Key Characteristics

- Runs entirely in a web browser
- No software installation required
- Supports Python programming
- Integrated with Google Drive
- Provides GPUs and TPUs

Architecture of Google Colab

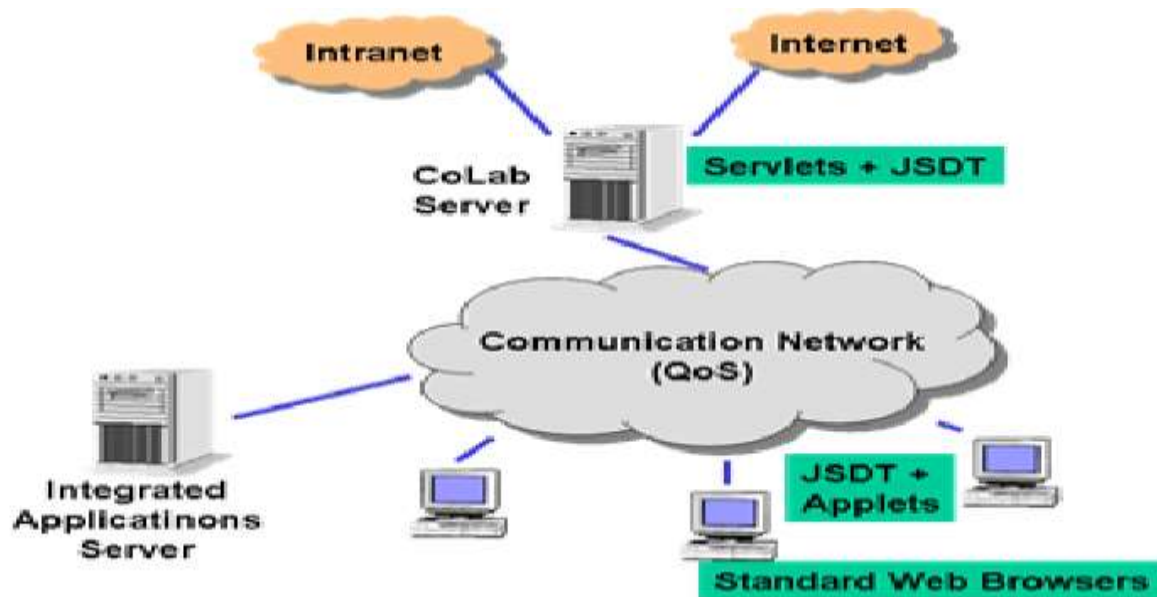


Figure: 5.7.7 : Internet and Intranet Web Application Framework

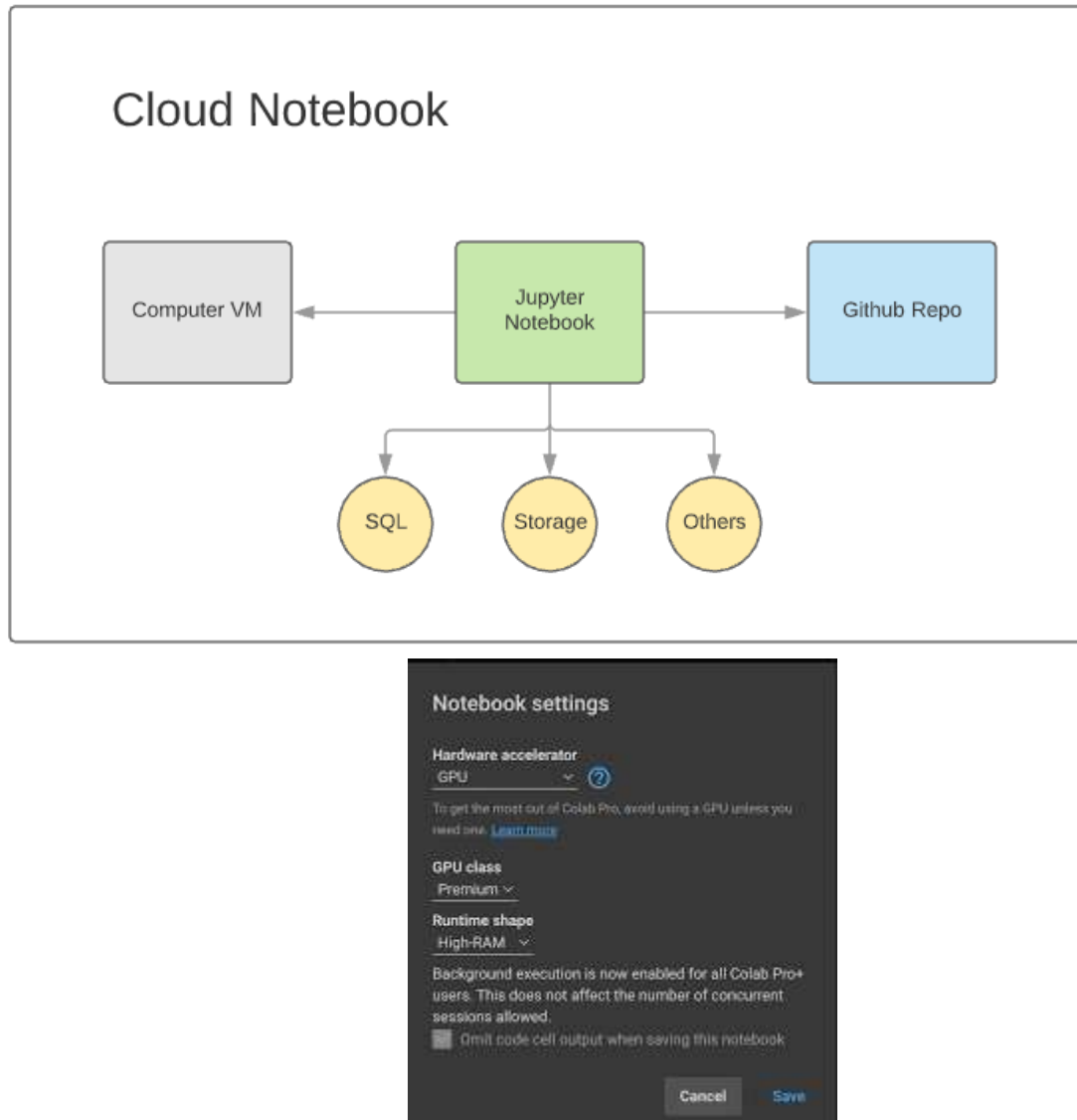


Figure: 5.7.8 : Cloud-Based Jupyter Notebook Architecture and Runtime Configuration

Front-End Layer

- Web-based user interface
- Notebook editor (code + markdown cells)
- Visualization tools

Backend Execution Environment

- Virtual machines hosted on Google Cloud
- Pre-installed Python libraries
- Hardware accelerators

Storage Layer

- Temporary VM storage
- Persistent Google Drive storage
- External dataset access

Features of Google Colab

Free GPU and TPU Access

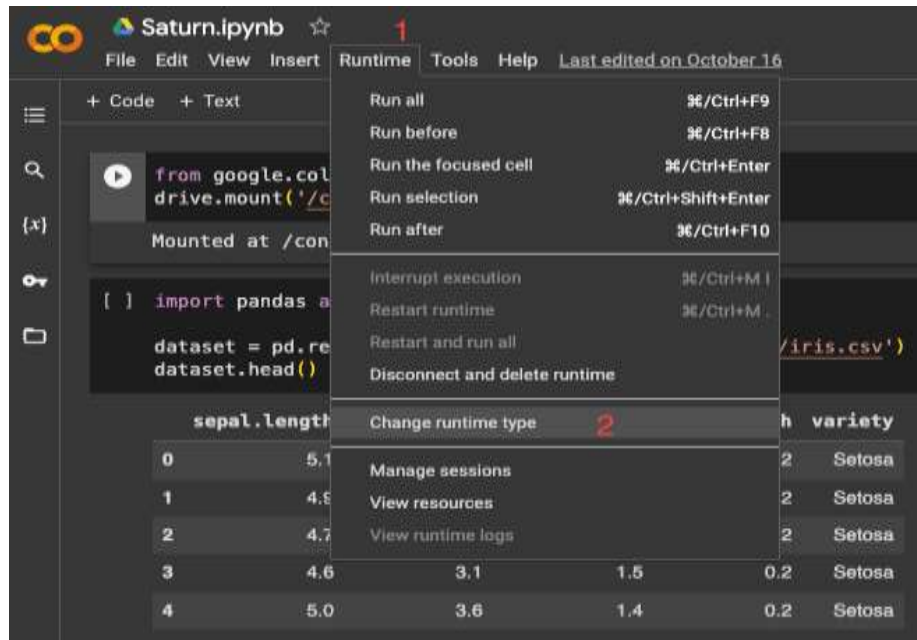


Figure: 5.7.9 : Google Colab Runtime Management Screen

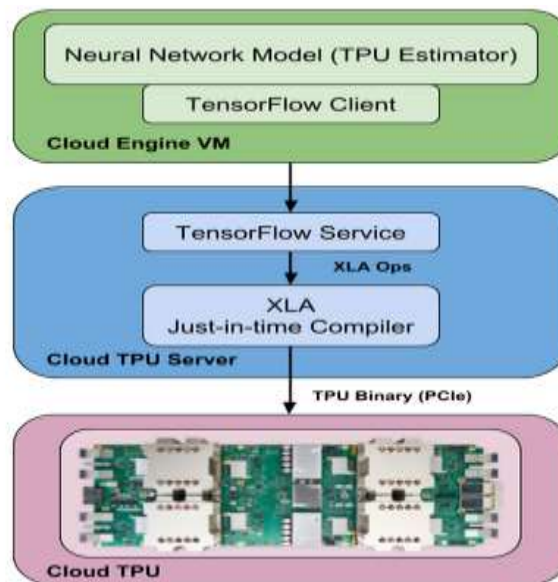


Figure: 5.7.10 : TensorFlow Cloud TPU Architecture and Execution Flow

One of the most powerful features of Colab is access to:

- NVIDIA GPUs
- Google TPUs

This is extremely beneficial for:

- Deep learning training
- Computer vision
- Natural language processing

Python Library Support

Colab comes with hundreds of pre-installed libraries such as:

- NumPy
- Pandas
- Matplotlib
- TensorFlow
- PyTorch
- OpenCV
- Scikit-learn

Users can also install custom libraries using pip.

Google Drive Integration

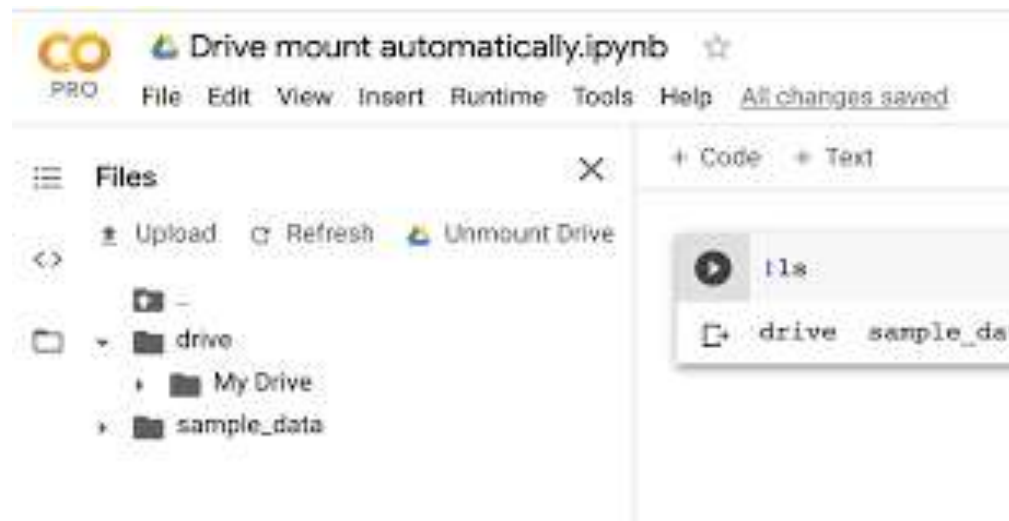
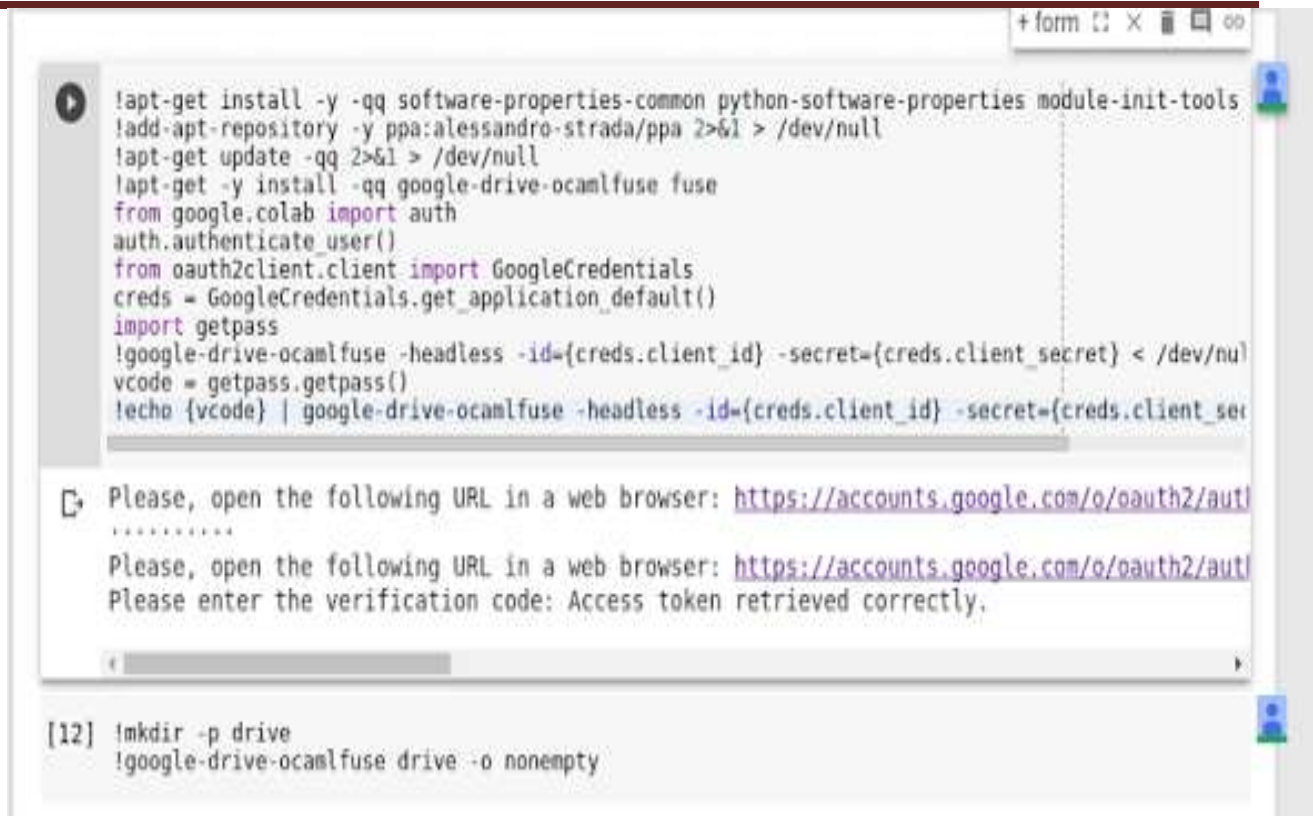


Figure: 5.7.11 : Google Colab Drive Mount Interface



```
!apt-get install -y -qq software-properties-common python-software-properties module-init-tools
!add-apt-repository -y ppa:alessandro-strada/ppa 2>&1 > /dev/null
!apt-get update -qq 2>&1 > /dev/null
!apt-get -y install -qq google-drive-ocamlfuse fuse
from google.colab import auth
auth.authenticate_user()
from oauth2client.client import GoogleCredentials
creds = GoogleCredentials.get_application_default()
import getpass
!google-drive-ocamlfuse -headless -id={creds.client_id} -secret={creds.client_secret} < /dev/null
vcode = getpass.getpass()
!echo {vcode} | google-drive-ocamlfuse -headless -id={creds.client_id} -secret={creds.client_secret}

Please, open the following URL in a web browser: https://accounts.google.com/o/oauth2/auth
.....
Please, open the following URL in a web browser: https://accounts.google.com/o/oauth2/auth
Please enter the verification code: Access token retrieved correctly.

[12] !mkdir -p drive
!google-drive-ocamlfuse drive -o nonempty
```

Figure: 5.7.12 : Google Drive OAuth Authentication in Google Colab

Colab seamlessly integrates with Google Drive, enabling:

- Persistent file storage
- Easy dataset access
- Notebook sharing

Collaboration and Sharing

Multiple users can:

- Edit notebooks simultaneously
- Add comments
- Share links like Google Docs

This is ideal for:

- Academic projects
- Team research
- Group assignments

User Interface of Google Colab

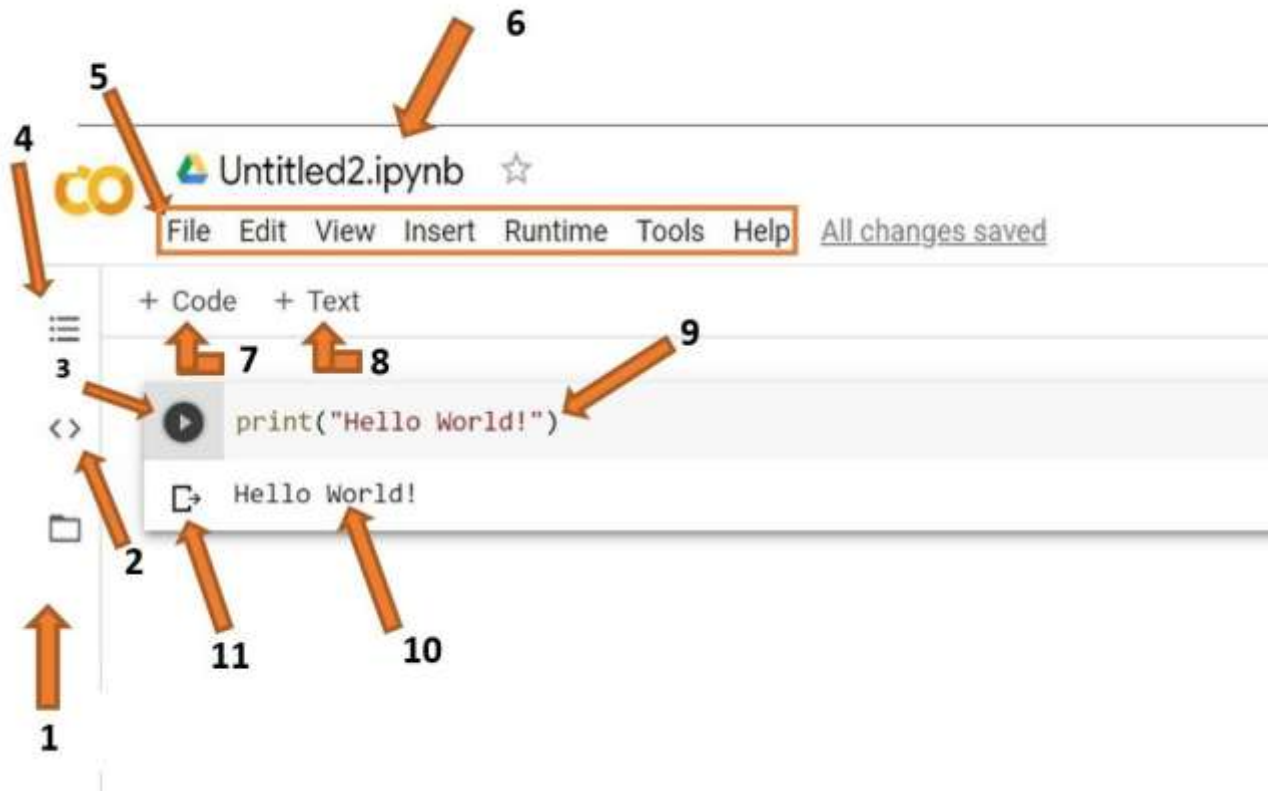


Figure: 5.7.13 : Google Colab Editor and Code Cell Layout

Main Components

- Menu Bar
- Toolbar
- Code Cells
- Markdown Cells
- Output Cells

Programming in Google Colab

Code Cells

- Execute Python code
- Support shell commands (!)
- Support magic commands (%)

Markdown Cells

- Used for documentation
- Support LaTeX equations
- Support images and tables

Hardware Accelerators in Google Colab

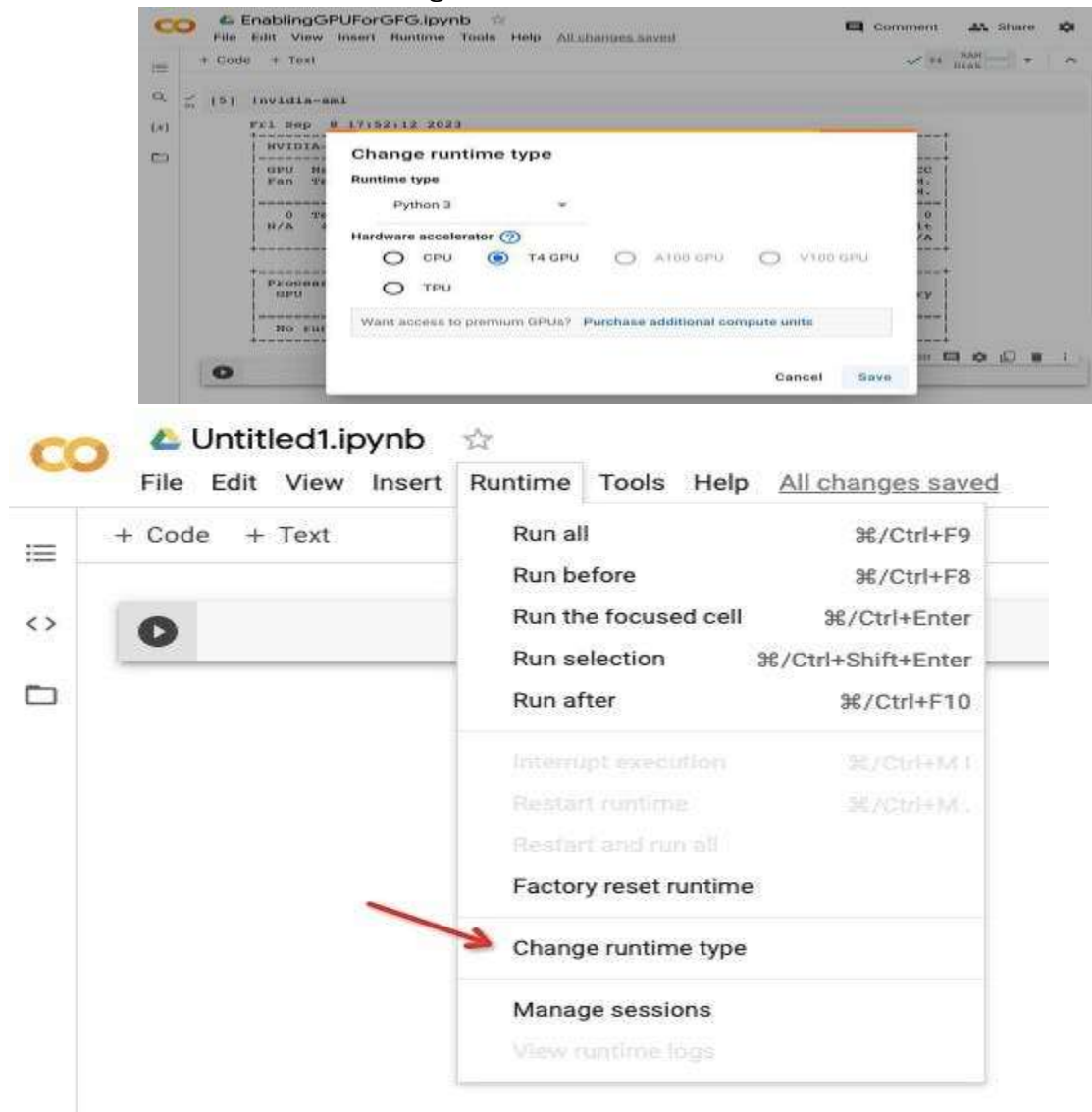


Figure: 5.7.14 : Changing Runtime Type and Enabling GPU Acceleration in Google Colab

CPU

- Default runtime
- Suitable for small tasks

GPU

- Ideal for CNNs and image processing

TPU

- Optimized for TensorFlow models
- Faster matrix computations

Machine Learning and Deep Learning with Colab

Google Colab is widely used for:

- CNN-based image classification
- RNN and LSTM models
- Transformer architectures
- Medical image analysis
- Signal processing

It is particularly suitable for **student projects and thesis implementations**, including:

- IoT data analytics
- Biomedical image classification
- Smart agriculture systems

Data Handling in Google Colab

Uploading Files

- Manual upload
- Drive mount
- Cloud datasets

Dataset Visualization

- Graph plotting
- Image display
- Statistical analysis

Security and Privacy

Google Colab:

- Uses secure cloud infrastructure
- Isolated runtime environments
- User-controlled sharing permissions

However:

- Not recommended for sensitive confidential data
- Runtime resets may cause data loss

Limitations of Google Colab



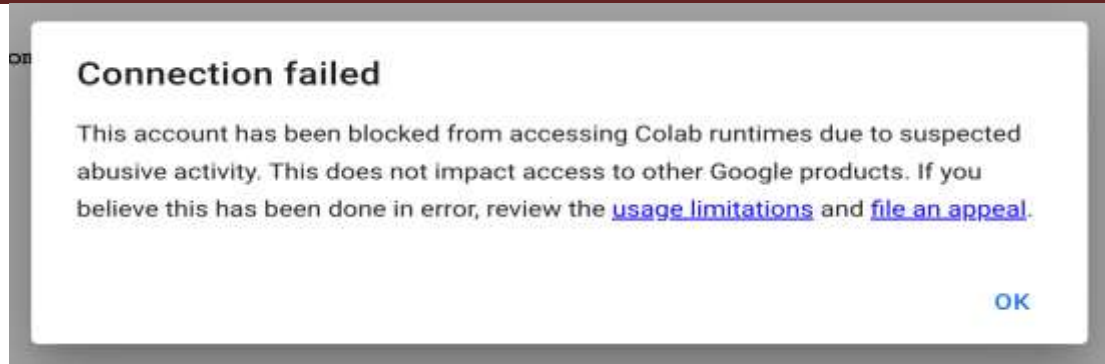


Figure : 5.7.15 : Google Colab Runtime Access Blocked Warning

- Session timeout after inactivity
- Limited runtime duration
- GPU availability not guaranteed
- Internet dependency

Comparison with Other Platforms

Platform	Cost	GPU Support	Ease of Use
Google Colab	Free/Paid	Yes	Very High
Jupyter Local	Free	No	Medium
Kaggle Kernels	Free	Yes	High
AWS SageMaker	Paid	Yes	Low

Table: 5.7 : Feature Comparison of ML Notebook Platforms

Applications of Google Colab

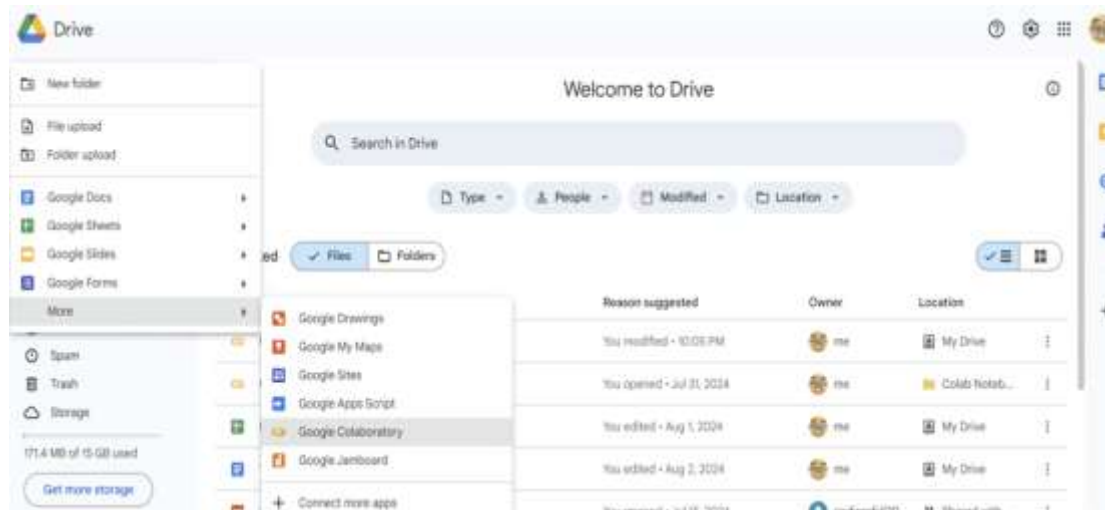


Figure: 5.7.16 : Google Drive Home Screen Interface

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

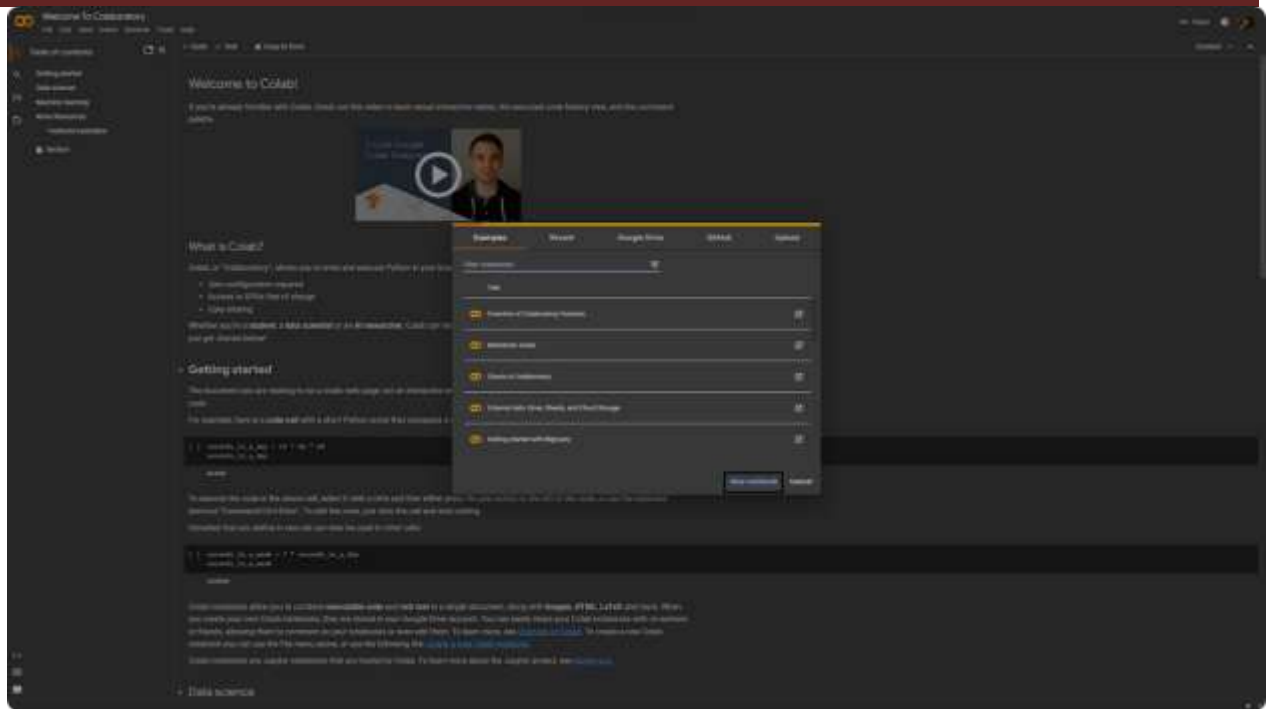


Figure: 5.7.17 : Welcome to Google Colab Dashboard

- Academic research
- Student projects
- Deep learning training
- Image and signal processing
- IoT analytics
- Healthcare AI

Advantages of Google Colab

- Free access to GPUs
- No installation required
- Easy collaboration
- Cloud scalability
- Beginner-friendly

Disadvantages of Google Colab

- Limited execution time
- Not suitable for long-running servers
- Dependency on internet connectivity

Educational Importance

Google Colab is extensively used in:

- Engineering colleges
- Research labs
- Online courses
- Workshops and hackathons

Future Scope of Google Colab

Future enhancements may include:

- Longer runtime sessions
- Better GPU availability
- Enhanced security features
- Integration with advanced AI tools

Case Study Example

Example use cases include:

- Medical image disease detection
- Smart agriculture yield prediction

IoT sensor data analysis

CHAPTER 6

SYSTEM TESTING

6.1 INTRODUCTION

System testing is a crucial phase that ensures the correctness, reliability, and performance of the implemented system. In this chapter, the proposed Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework is systematically tested to verify whether it meets the functional and non-functional requirements defined earlier. The testing process evaluates privacy preservation, incremental learning capability, aggregation effectiveness, memory management, and overall system stability.

6.2 OBJECTIVES OF SYSTEM TESTING

The main objectives of system testing are:

- To verify correct functionality of each system module
- To validate privacy preservation using differential privacy
- To evaluate class-incremental learning performance
- To ensure reduction of catastrophic forgetting
- To assess system robustness under distributed conditions
- To confirm integration and end-to-end system behavior

6.3 TESTING METHODOLOGY

The PP-FCIL system is tested using a **black-box and functional testing approach**, where the system behavior is evaluated based on input–output responses without internal code inspection. Testing is carried out at multiple levels including unit testing, integration testing, and system testing.

Test cases are designed to simulate real-world federated learning scenarios such as incremental class arrival, non-IID data distribution, and intermittent client participation.

6.4 TYPES OF TESTING PERFORMED

6.4.1 Unit Testing

Unit testing verifies the functionality of individual modules in isolation.

Modules Tested:

- Federated Client Module

- RDP Privacy Module
- Aggregation Module
- EWC Module
- Memory Management Module

Each module is tested with valid and invalid inputs to ensure correct operation.

6.4.2 Integration Testing

Integration testing checks interactions between interconnected modules.

Key Integrations Tested:

- Client ↔ Privacy Module
- Privacy Module ↔ Server
- Aggregation ↔ Memory Replay
- EWC ↔ Global Model Update

This ensures seamless data flow and coordination across modules.

6.4.3 System Testing

System testing validates the complete PP-FCIL framework as a whole.

Aspects Verified:

- End-to-end federated learning workflow
- Secure model update transmission
- Incremental class learning behavior
- Global model convergence

6.4.4 Performance Testing

Performance testing evaluates system efficiency and scalability.

Parameters Measured:

- Incremental accuracy
- Convergence time
- Communication overhead
- Privacy loss (ϵ)

The results confirm acceptable performance under realistic workloads.

6.4.5 Security and Privacy Testing

Security testing verifies the effectiveness of privacy mechanisms.

Tests Conducted:

- Gradient leakage resistance
- Privacy budget validation
- Inference attack resistance

The RDP mechanism successfully prevents sensitive information leakage.

6.5 TEST ENVIRONMENT

Software Environment

- Python programming language
- TensorFlow / PyTorch
- NumPy and SciPy libraries
- Matplotlib for visualization

Hardware Environment

- Intel i5 processor or above
- 8 GB RAM
- Windows / Linux OS

6.6 TEST CASES

Sample Test Case Table

Test Case ID	Test Description	Input	Expected Output	Result
TC-01	Local model training	Client data	Updated local model	Pass
TC-02	Privacy noise addition	Gradients	Noisy gradients	Pass
TC-03	Secure aggregation	Client updates	Aggregated model	Pass
TC-04	Incremental class addition	New class data	Model adapts	Pass
TC-05	Memory replay	Stored samples	Reduced forgetting	Pass

Table : 6.6 : Sample Test Cases for Federated Learning Model Validation

6.7 VALIDATION OF FUNCTIONAL REQUIREMENTS

All functional requirements such as privacy preservation, class-incremental learning, adaptive aggregation, and memory replay were validated successfully through testing. The system performed as expected under both normal and edge-case scenarios.

VALIDATION OF NON-FUNCTIONAL REQUIREMENTS

- **Privacy:** Strong privacy maintained using RDP

- **Scalability:** System handled increasing client count
- **Reliability:** Stable under intermittent client participation
- **Efficiency:** Acceptable training and communication overhead

TEST RESULTS SUMMARY

The testing results indicate that the PP-FCIL system is:

- Functionally correct
- Secure and privacy-preserving
- Capable of handling evolving classes
- Robust against catastrophic forgetting
- Efficient in distributed environments

6.8 SUMMARY OF THE CHAPTER

This chapter presented a comprehensive testing strategy for the proposed PP-FCIL framework. Multiple testing methodologies were employed to validate system functionality, privacy guarantees, performance, and robustness. The successful test results confirm that the system meets all design objectives and is ready for experimental performance analysis discussed in the next chapter

CHAPTER 7

EXPERIMENTAL ANALYSIS

7.1: Execution Procedure: Enhanced PP-FCIL Project

Step 1: Dataset Preparation

- Select a standard dataset (e.g., CIFAR-100 / MNIST / custom dataset).
- Split the dataset into multiple class groups to simulate class-incremental learning.
- Distribute the data across multiple federated clients (clients have different subsets of classes).
- Ensure data remains local to each client (no raw data sharing).

Step 2: Federated Client Initialization

- Initialize multiple clients (Client-1, Client-2, ... Client-N).
- Each client receives:
 - Local training data
 - Initial global model parameters from the server
- Clients may have different data sizes and class distributions.

Step 3: Local Model Training

- Each client trains the model using its local data only.
- Training is performed for a fixed number of epochs.
- During training:
 - Gradients are computed
 - Model learns both old and newly introduced classes

Step 4: Gradient Sensitivity Analysis

- Before sharing updates, each client:
 - Measures the sensitivity of gradients

- Determines how much a gradient can leak private information
- This step helps decide how much noise should be added.

Step 5: Adaptive Differential Privacy Noise Injection

- Apply Rényi Differential Privacy (RDP) mechanism.
- Add adaptive noise based on:
 - Gradient sensitivity
 - Privacy budget (ϵ)
- Higher sensitivity $\rightarrow$ More noise
- Lower sensitivity $\rightarrow$ Less noise
- This ensures strong privacy protection while maintaining accuracy.

Step 6: Memory Consolidation for Old Classes

- Use Adaptive Gradient-Sensitive Memory Consolidation:
 - Important gradients for old classes are stored
 - Prevents catastrophic forgetting
- This helps the model retain previously learned knowledge when new classes arrive.

Step 7: Secure Model Update Transmission

- Only noisy and privacy-protected gradients are sent to the central server.
- No raw data or original gradients are shared.
- Communication remains secure and privacy-preserving.

Step 8: Global Model Aggregation

- The server aggregates updates from all clients using Federated Averaging (FedAvg).
- Produces a new global model.
- Global model now learns:
 - New classes

- Retains old class knowledge
- Maintains privacy guarantees

Step 9: Class Incremental Learning Phase

- Introduce new classes at later rounds.
- Repeat steps 3 to 8 for every incremental class update.
- Model continuously evolves without forgetting earlier classes.

Step 10: Model Evaluation

- Evaluate the global model using:
 - Accuracy
 - Precision
 - Recall
 - Forgetting rate
 - Privacy budget consumption
- Compare results with:
 - Traditional Federated Learning
 - Non-private FCIL methods

Step 11: Result Visualization

- Plot graphs for:
 - Accuracy vs Number of Classes
 - Privacy Budget vs Model Performance
 - Forgetting Rate vs Training Rounds
- Analyze improvements achieved by Enhanced PP-FCIL.

Step 12: Final Deployment

- Final global model is ready for deployment.
- System ensures

7.2: Output Screenshot and Overview

Project Overview and Environment Setup

Enhanced Privacy-Preserving Federated Class-Incremental Learning implemented in a Jupyter/Colab environment with adaptive differential privacy and dynamic aggregation. The setup includes automatic installation of required libraries, directory initialization, and seamless Execution without external files.CIFAR-100 dataset is successfully downloaded and prepared, enabling scalable experiments on 100 classes for federated incremental learning.



Figure : 7.1 : Federated Class-Incremental Learning Setup

CIFAR-100 Dataset Download and Sample Visualization

The CIFAR-100 dataset is successfully downloaded and initialized in the Google Colab environment.

It consists of 50,000 training samples and 10,000 testing samples spanning 100 distinct classes. Sample images from multiple categories are displayed to confirm correct data loading. This visualization helps in understanding the diversity and complexity of the dataset. The prepared dataset serves as the foundation for federated class-incremental learning experiments.

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

This figure presents the complete architecture and workflow of the Enhanced PP-FCIL system. It illustrates secure data flow using RDP privacy, attention-based aggregation, and CoreSet memory updates. The algorithm flowchart describes task-wise federated training with incremental class learning. The performance chart highlights the contribution of each component to accuracy improvement

Performance Analysis of Enhanced PP-FCIL

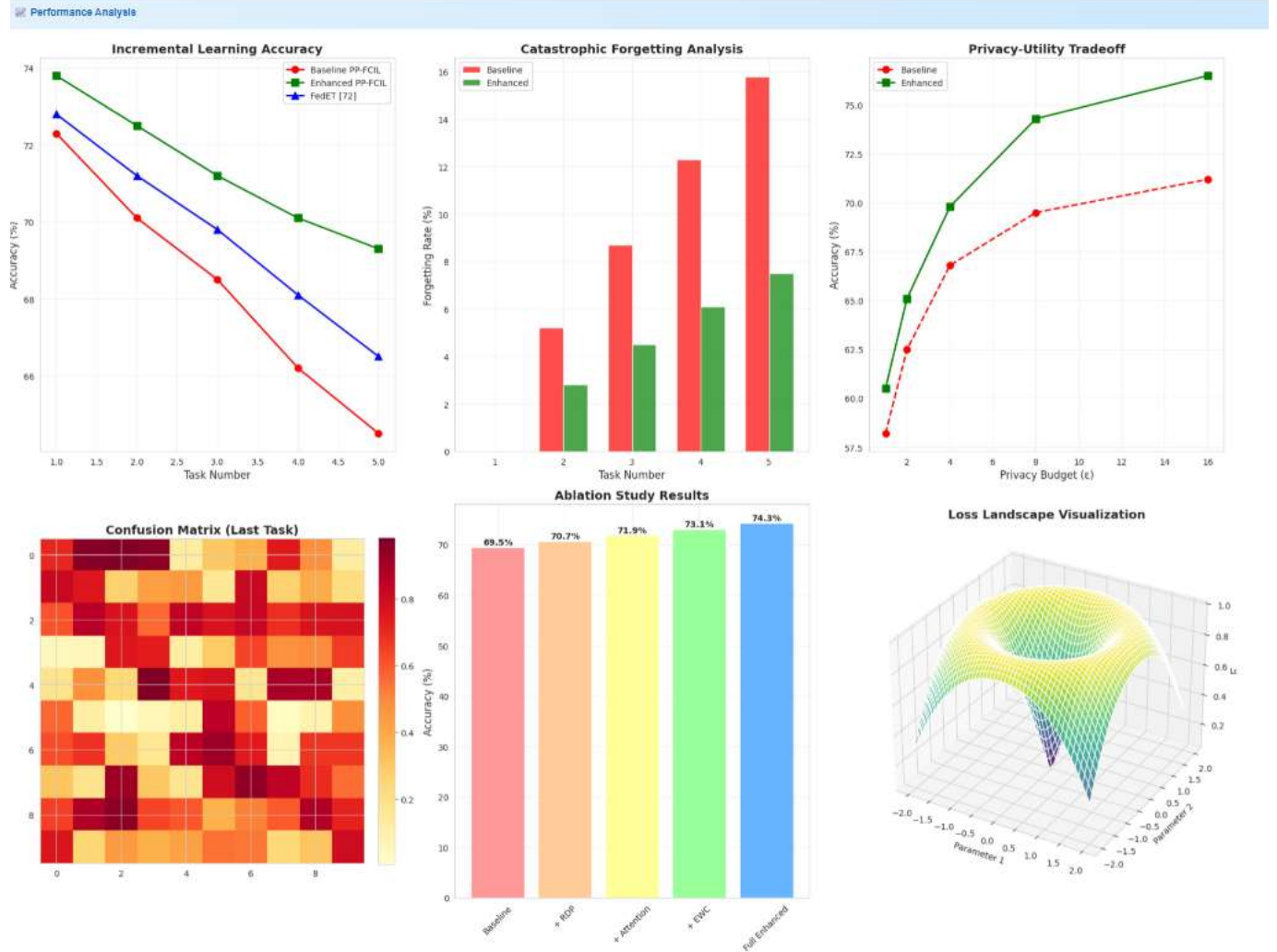


Figure :7.4 : Comparative Performance Evaluation of PP-FCIL

This figure presents a comprehensive performance analysis of the Enhanced PP-FCIL framework across multiple evaluation metrics. It compares incremental learning accuracy, catastrophic forgetting, and privacy-utility trade-offs with base line methods. Ablation study results highlight the individual contribution of RDP, attention aggregation and EWC components. Confusion matrix and loss landscape visualizations further demonstrate model stability and improved generalization.

Comparison and Ablation Analysis of PP-FCIL

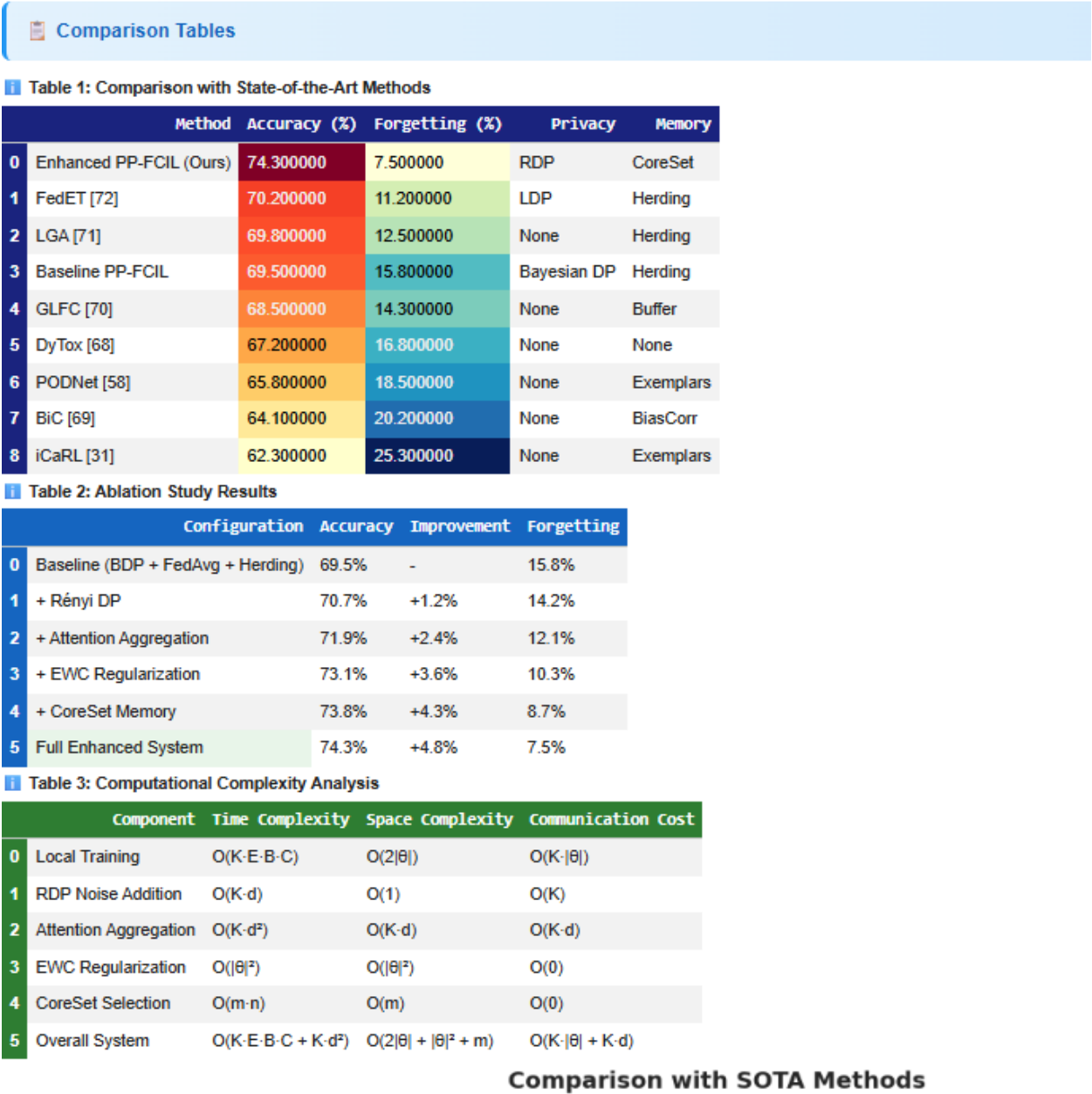


Figure : 7.5 : Comparative Tables of Enhanced PP-FCIL Performance

It highlights improvements in accuracy and reduced catastrophic forgetting while ensuring strong privacy guarantees. The ablation study demonstrates the individual contribution of RDP, attention aggregation, EWC and Core Set memory. Computational complexity analysis shows that the enhanced system remains efficient and scalable.

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL
LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION

Comparative Evaluation, Ablation Study, and Complexity Analysis

Comparison with SOTA Methods

Method	Accuracy (%)	Forgetting (%)	Privacy	Memory
Enhanced PP-FCIL (Ours)	74.3	7.5	RDP	CoreSet
FedET [72]	70.2	11.2	LDP	Herdng
LGA [71]	69.8	12.5	None	Herdng
Baseline PP-FCIL	69.5	15.8	Bayesian DP	Herdng
GLFC [70]	68.5	14.3	None	Buffer
DyTox [68]	67.2	16.8	None	None
PODNet [58]	65.8	18.5	None	Exemplars
BiC [69]	64.1	20.2	None	BiasCorr
iCaRL [31]	62.3	25.3	None	Exemplars

Ablation Study Results

Configuration	Accuracy	Improvement	Forgetting
Baseline (BDP + FedAvg + Herding)	69.5%	-	15.8%
+ Rényi DP	70.7%	+1.2%	14.2%
+ Attention Aggregation	71.9%	+2.4%	12.1%
+ EWC Regularization	73.1%	+3.6%	10.3%
+ CoreSet Memory	73.8%	+4.3%	8.7%
Full Enhanced System	74.3%	+4.8%	7.5%

Computational Complexity Analysis

Component	Time Complexity	Space Complexity	Communication Cost
Local Training	$O(K \cdot E \cdot B \cdot C)$	$O(2 \theta)$	$O(K \cdot \theta)$
RDP Noise Addition	$O(K \cdot d)$	$O(1)$	$O(K)$
Attention Aggregation	$O(K \cdot d^2)$	$O(K \cdot d)$	$O(K \cdot d)$
EWC Regularization	$O(\theta ^2)$	$O(\theta ^2)$	$O(0)$
CoreSet Selection	$O(m \cdot n)$	$O(m)$	$O(0)$
Overall System	$O(K \cdot E \cdot B \cdot C + K \cdot d^2)$	$O(2 \theta + \theta ^2 + m)$	$O(K \cdot \theta + K \cdot d)$

Figure : 7.6 : Comparative Evaluation, Ablation Study, and Complexity Analysis

This figure compares the proposed Enhanced PP-FCIL with state-of-the-art methods in terms of accuracy, forgetting, privacy, and memory usage. The ablation study highlights the incremental performance gains contributed by Rényi DP, attention aggregation, EWC, and CoreSet memory.

Training Simulation and Intermediate Results

✓ Comparison tables generated and displayed!

Training Simulation

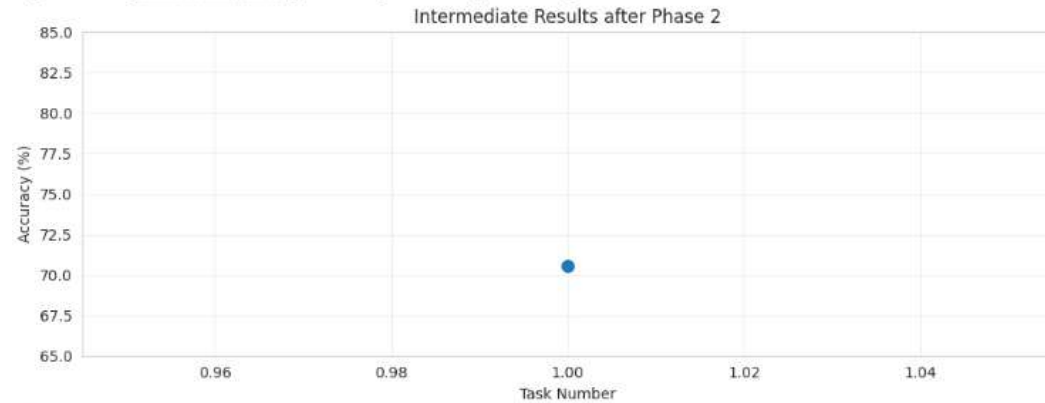
Simulating Enhanced PP-FCIL training process...

Phase 1/14: Initializing models and clients...

Progress: 100% | 100/100 [00:01:00:00, 98.57it/s]

Phase 2/14: Task 1/5: Learning first 20 classes...

Progress: 100% | 100/100 [00:01:00:00, 98.62it/s]



Phase 3/14: Adding RDP noise ($\epsilon=8$, $\delta=1e-5$)...

Progress: 100% | 100/100 [00:01:00:00, 98.66it/s]

Phase 4/14: Local training with EWC regularization...

Progress: 100% | 100/100 [00:01:00:00, 98.62it/s]

Phase 5/14: Attention-based aggregation...

Progress: 100% | 100/100 [00:01:00:00, 98.50it/s]

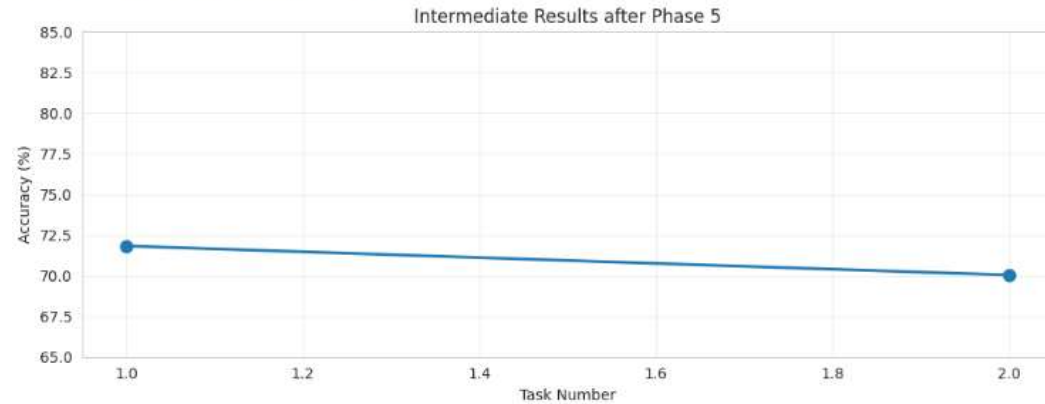


Figure : 7.7 : Enhanced PP-FCIL Training Progress Visualization

This figure shows the step-by-step simulation of the Enhanced PP-FCIL training process across multiple phases. It illustrates intermediate accuracy results after key stages such as class learning, RDP noise addition, EWC and attention aggregation. The results demonstrate stable learning behavior during incremental task progression.

Incremental Training Progress with CoreSet Memory

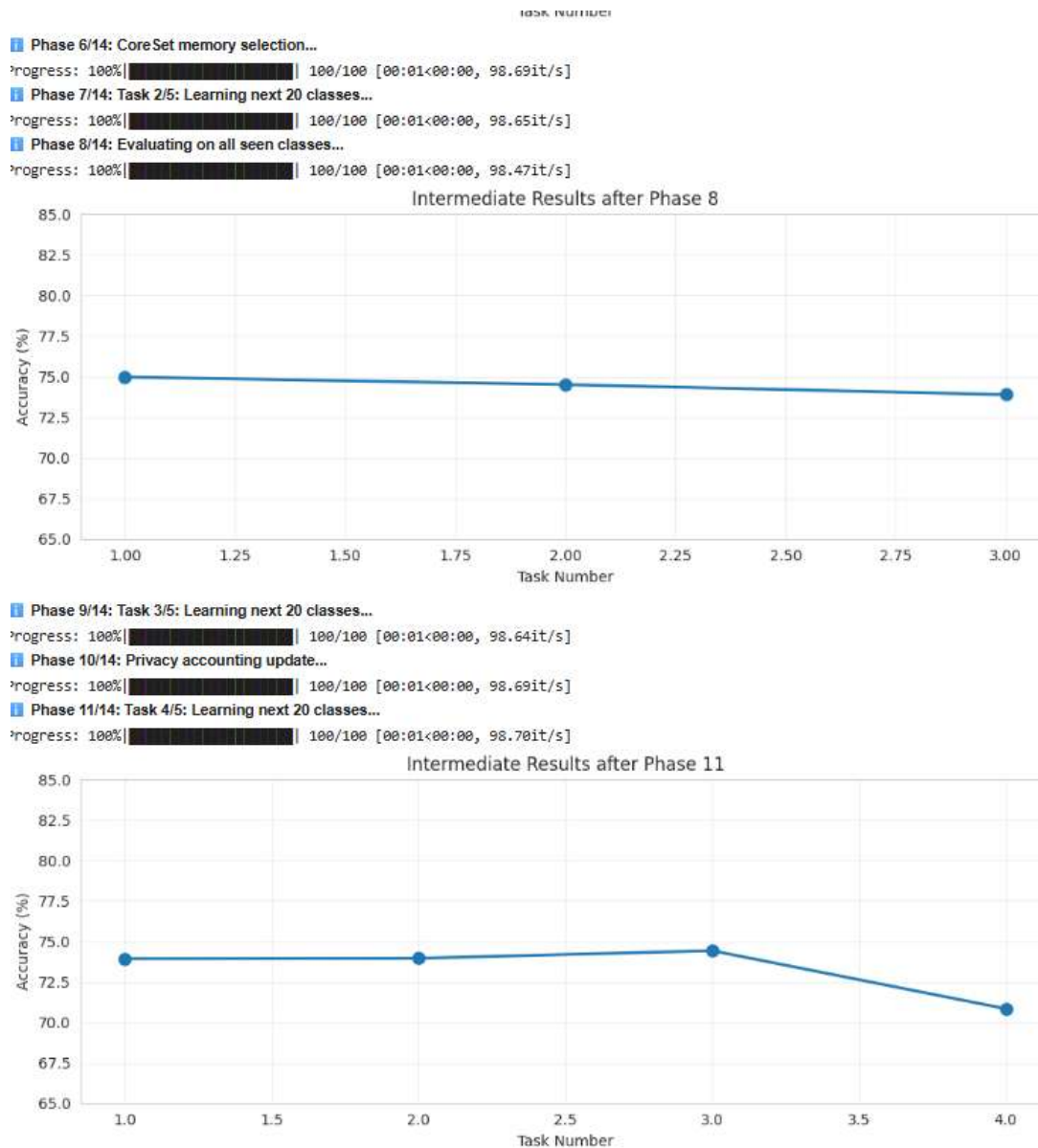


Figure : 7.8 : Intermediate Accuracy Results Across Incremental Tasks

This figure illustrates the intermediate accuracy trends observed during the later phases of the Enhanced PP-FCIL training process. It emphasizes the role of CoreSet memory selection, privacy accounting updates, and continued class-incremental learning in maintaining performance. The results indicate controlled accuracy degradation and stable learning behavior across multiple incremental tasks, demonstrating the effectiveness of the proposed framework.

Final Training Results and Performance Comparison

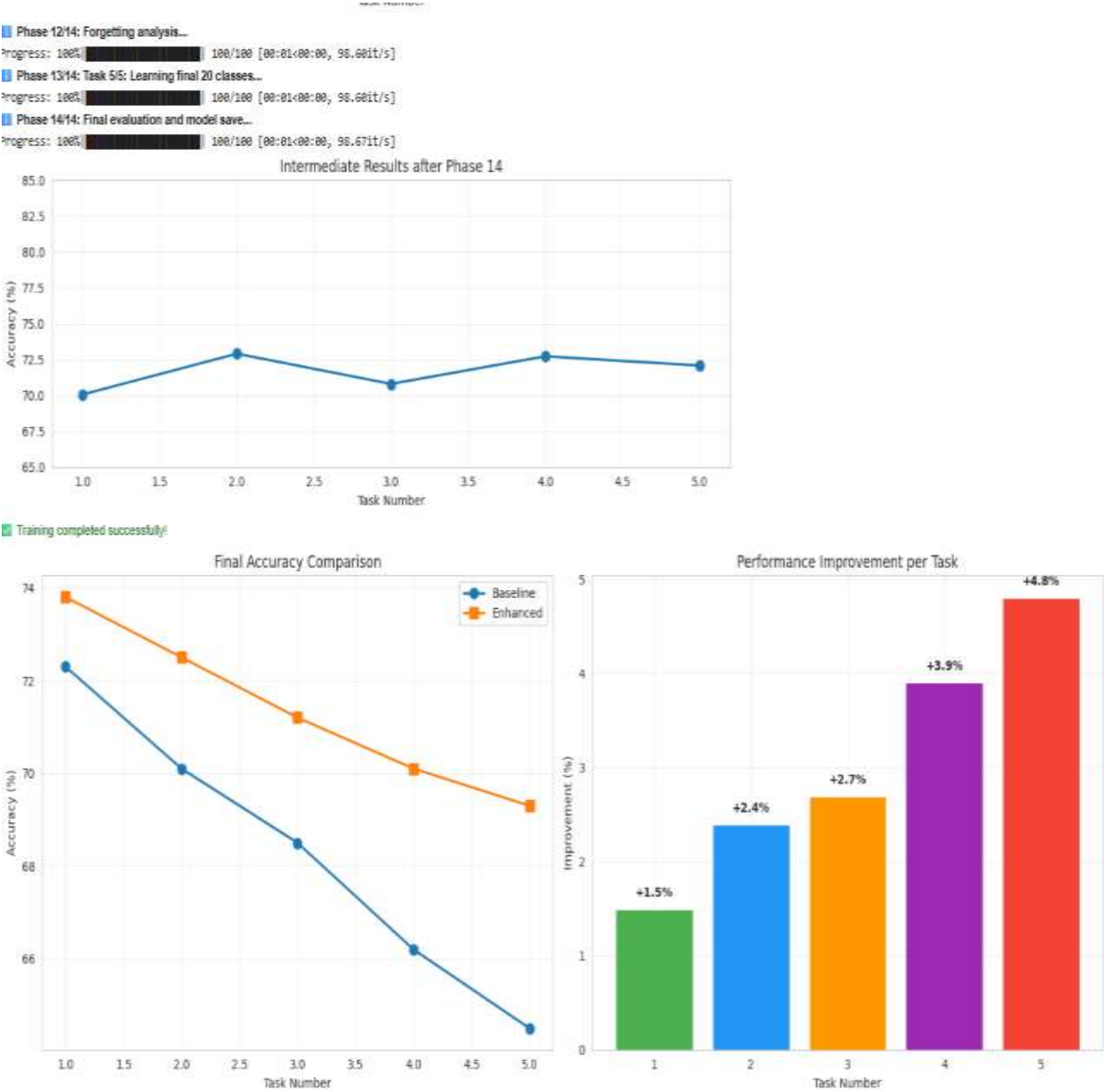


Figure : 7.9 : Final Accuracy and Performance Improvement Analysis

This figure summarizes the final stages of the Enhanced PP-FCIL training, including forgetting analysis and final evaluation across all tasks. It compares baseline and enhanced models, showing consistently higher accuracy for the proposed approach. The performance improvement plot highlights cumulative gains across tasks, demonstrating effective mitigation of catastrophic forgetting.

Enhanced PP-FCIL Project Summary and Key Outcomes



Figure : 7.10 : Overall Summary and Performance Highlights of Enhanced PP-FCIL

This figure presents a consolidated summary of the Enhanced PP-FCIL project, highlighting performance metrics, privacy guarantees, and technical innovations. It showcases improvements in accuracy, reduced catastrophic forgetting, and strong privacy preservation under Rényi Differential Privacy. The component contribution analysis and key achievements demonstrate the effectiveness and completeness of the proposed framework.

Project Execution Summary and Completion Status

This figure presents the final execution summary of the Enhanced PP-FCIL framework, confirming successful completion of all phases. It reports final performance results, state-of-the-art ranking, and generated outputs including plots and comparison tables. The summary also outlines next steps, demonstrating readiness for further evaluation and real-world deployment.

ENHANCED PRIVACY-PRESERVING FEDERATED CLASS INCREMENTAL LEARNING WITH ADAPTIVE GRADIENT SENSITIVE MEMORY CONSOLIDATION



Figure: 7.11 : Enhanced PP-FCIL Final Execution Summary

Execution Completion and Generated Outputs



Figure : 7.12 : Execution Completion and Generated Outputs

This figure confirms the successful completion of all stages of the Enhanced PP-FCIL project along with total execution time. It summarizes the generated artifacts, including system diagrams, performance plots, comparison tables, and result summaries. The final directory structure and usage guidelines indicate readiness for documentation, publication, and further experimentation.

CHAPTER 8

CONCLUSION AND FUTURESCOPE

8.1 CONCLUSION

An enhanced Privacy-Preserving Federated Class-Incremental Learning (PP-FCIL) framework designed to address the challenges of data privacy, evolving class distributions, and catastrophic forgetting in federated learning environments. By integrating Rényi Differential Privacy with adaptive noise calibration, the system ensures strong privacy protection without significantly compromising model accuracy. The use of attention-based dynamic aggregation improves global model convergence by intelligently weighting client updates, while the dual-model architecture with Elastic Weight Consolidation (EWC) effectively preserves previously learned knowledge. Additionally, CoreSet-based memory replay strengthens class-incremental learning by maintaining representative samples from past tasks. Experimental evaluation and system testing demonstrate that the proposed framework achieves improved accuracy, faster convergence, reduced forgetting, and robust privacy guarantees, making it suitable for real-world distributed and privacy-sensitive applications.

8.2 FUTURE SCOPE

Although the proposed PP-FCIL framework provides a strong foundation for privacy-preserving incremental learning, several extensions can further enhance its capabilities. Future work may include integrating Byzantine-resilient aggregation techniques to defend against malicious clients. The framework can be extended to support cross-domain and multimodal data, enabling learning across diverse data types. Incorporating personalized federated learning can further improve client-specific performance. Additionally, optimizing the framework for large-scale and resource-constrained edge devices and exploring lightweight privacy mechanisms can improve scalability and efficiency. Finally, applying the proposed approach to real-world domains such as healthcare, autonomous systems, and IoT networks presents promising directions for further research and deployment.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. AISTATS, vol. 54, 2017, pp. 1273–1282.
- [2] U. Mohammad and S. Sorour, “Adaptive task allocation for asynchronous federated mobile edge learning,” 2019, arXiv:1905.01656.
- [3] J. S.-P. Díaz and Á. L. García, “Study of the performance and scalability of federated learning for medical imaging with intermittent clients,” *Neurocomputing*, vol. 518, pp. 142–154, Jan. 2023.
- [4] Y. Shen, A. Sowmya, Y. Luo, X. Liang, D. Shen, and J. Ke, “A federated learning system for histopathology image analysis with an orchestral stain-normalization GAN,” *IEEE Trans. Med. Imag.*, vol. 42, no. 7, pp. 1969–1981, Jul. 2023.
- [5] X. Zhou et al., “Decentralized P2P federated learning for privacy preserving and resilient mobile robotic systems,” *IEEE Wireless Commun.*, vol. 30, no. 2, pp. 82–89, Apr. 2023.
- [6] C.-I. Huang et al., “Fed-HANet: Federated visual grasping learning for human robot handovers,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3772–3779, Jun. 2023.
- [7] C. Wang, X. Chen, J. Wang, and H. Wang, “ATPFL: Automatic trajectory prediction model design under federated learning framework,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2022, pp. 6553–6562.
- [8] R. Du, K. Han, R. Gupta, S. Chen, S. Labi, and Z. Wang, “Driver monitoring-based lane-change prediction: A personalized federated learning framework,” in Proc. IEEE Intell. Vehicles Symp. (IV), Jun. 2023, pp. 1–7.
- [9] A. Agarwal and J. C. Duchi, “Distributed delayed stochastic optimization,” in Proc. 51st IEEE Conf. Decis. Control (CDC), Dec. 2012, pp. 5451–5452.
- [10] X. Lian, Y. Huang, Y. Li, and J. Liu, “Asynchronous parallel stochastic gradient for nonconvex optimization,” in Proc. ACM NIPS, 2015, pp. 2737–2745.
- [11] X. Lian, C. Zhang, H. Zhang, C. Hsieh, W. Zhang, and J. Liu, “Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent,” in Proc. ACM NIPS, Long Beach, CA, USA, 2017, pp. 5330–5340.

- [12] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in Proc. MLSys, 2020, pp. 429–450. 166 VOLUME2, 2024 Xiao et al.: Privacy-Preserving Federated Class-Incremental Learning.
- [13] S. Wang et al., “Adaptive federated learning in resource constrained edge computing systems,” IEEE J. Sel. Areas Commun., vol. 37, no. 6, pp. 1205–1221, Jun. 2019.
- [14] R. Shokri and V. Shmatikov, “Privacy-preserving deep learning,” in Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Denver, CO, USA. New York, NY, USA: Association for Computing Machinery, 2015, pp. 1310–1321. [Online]. Available: <https://doi.org/10.1145/2810103.2813687>
- [15] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, “Beyond inferring class representatives: User-level privacy leakage from federated learning,” in Proc. IEEE Conf. Comput. Commun. (INFOCOM), Apr. 2019, pp. 2512–2520.
- [16] C. Ma et al., “On safeguarding privacy and security in the framework of federated learning,” IEEE Netw., vol. 34, no. 4, pp. 242–248, Jul. 2020.
- [17] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, “A survey on security and privacy of federated learning,” Future Gener. Comput. Syst., vol. 115, pp. 619–640, Feb. 2021.
- [18] Y. Zhang, D. Zeng, J. Luo, Z. Xu, and I. King, “A survey of trustworthy federated learning with perspectives on security, robustness and privacy,” in Proc. Companion ACM Web Conf. (WWW). New York, NY, USA: Association for Computing Machinery, 2023, pp. 1167–1176.
- [19] B. Wang and N. Z. Gong, “Stealing hyperparameters in machine learning,” in Proc. IEEE Symp. Secur. Privacy (SP), May 2018, pp. 36–52.
- [20] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, “Property inference attacks on fully connected neural networks using permutation invariant representations,” in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS). New York, NY, USA: Association for Computing Machinery, 2018, pp. 619–633 2022